## Содержание


Введение ..... 4
1 Описание предметной области ..... 6
1.1 Видение проекта ..... 6
1.2 Математические (вербальные) модели ..... 8
2 Проектирование программного модуля ..... 12
2.1 Перечень функциональности ..... 12
2.2 Сценарии взаимодействия ..... 14
2.3 Прототипирование интерфейса взаимодействия с пользователем (графического интерфейса). ..... 17
2.4 Составление индивидуального плана работы ..... 20
3. Разработка документации программного модуля ..... 23
3.1 Диаграмма классов ..... 23
3.2 Диаграмма вариантов использования ..... 30
3.3 Диаграммы последовательностей ..... 33
4. Разработка документации приложения ..... 37
4.1 Выбор и обоснование технологий, используемых в программном модуле ..... 37
4.2 Описание используемых структур данных ..... 38
5 Тестирование приложения ..... 43
Заключение ..... 48
Список литературы ..... 49
Приложение 1 – Код программных модулей ..... 50

## Введение

Объект автоматизации — веб-приложение «Умная парковка»: модуль «отображение свободных и занятых мест». Он входит в общую систему мониторинга и управления парковкой, но в индивидуальном задании рассматривается как отдельный контур получения, обработки и визуализации данных о доступности парковочных мест на карте с учётом ограничений и актуального состояния парковки.

Назначение. Для пользователей — интерактивная карта парковки с отображением свободных и занятых мест, быстрый выбор свободного слота, просмотр статуса выбранного места и переход к бронированию. Для администратора — контроль доступности мест и корректности отображаемых статусов: установка ручных статусов (ремонт, резерв, закрыто), отключение и включение мест, а также согласование данных карты с результатами мониторинга и активными бронированиями.

Применение. Единая карта мест в веб-интерфейсе (личный кабинет пользователя и админ-панель); REST API (FastAPI) и хранение данных в SQLite; получение фактической занятости из модуля мониторинга (камера, обработка кадра нейросетью) и объединение с логическими статусами (активные брони, ручные ограничения через таблицу slot_overrides). Данные выдаются клиенту в формате JSON и отображаются в виде сетки мест с цветовой индикацией. Разграничение прав по ролям (user, admin) определяет доступ к операциям изменения статусов мест и административным функциям.

Практическая значимость — актуализация сведений о доступности парковочных мест для сокращения времени поиска свободного места и снижения нагрузки на парковку, повышение наглядности состояния парковки для пользователей, а также единая техническая связка «браузер — API — SQLite» с возможностью расширения аналитикой и административными инструментами командного проекта.

Цель (индивидуальная часть) — реализовать в «Умной парковке» модуль отображения свободных и занятых мест: обеспечить получение и выдачу актуального состояния слотов, корректное отображение на карте с учётом бизнес-правил (занятость по мониторингу, активные брони, ручные статусы), а также проверить работоспособность сквозными сценариями.

Для достижения поставленной цели были определены следующие задачи:

— изучить предметную область и сформулировать требования к модулю отображения свободных и занятых мест (с учётом границ индивидуального задания);

— спроектировать модель данных и REST API для выдачи карты мест и ручных статусов доступности;

— реализовать backend и frontend для отображения карты мест в веб-приложении и обновления данных по запросу;

— организовать контроль прав доступа к административным операциям (ручные статусы мест, отключение/включение) и обеспечить корректное объединение фактической занятости с логическими ограничениями;

— протестировать модуль: сценарии просмотра карты, выбора свободного места, проверка корректности статусов (свободно/занято/закрыто), права администратора на изменение доступности мест и выполнение типовых запросов к API в согласованные сроки отклика.

# 1 Описание предметной области

## 1.1 Видение проекта

Подсистема «Отображение свободных и занятых мест» предназначена для поддержки мониторинга загруженности парковки и информирования пользователей о доступности мест в режиме, близком к реальному времени. Основная идея — объединить в одном интерфейсе фактическую занятость парковочных слотов (по данным мониторинга) и логические ограничения (активные брони, ручные статусы администратора), чтобы пользователь мог быстро найти свободное место и выбрать его для бронирования, а администрация — контролировать доступность инфраструктуры и корректность отображаемых статусов на карте.

Ключевые функции подсистемы:

— Отмена брони в один клик. В личном кабинете у каждой активной брони есть кнопка отмены. Пользователь нажимает её один раз — заявка снимается, место снова считается доступным для выбора на карте, если на него не действуют другие ограничения.

— Тарифный калькулятор. При указании длительности стоянки система автоматически рассчитывает стоимость по действующему почасовому тарифу и показывает ориентировочную сумму рядом с формой бронирования, чтобы пользователь видел цену до подтверждения.

— Продление оплаченного времени. Для активной брони доступна команда продления (например, на один час). Система увеличивает время окончания парковки, при необходимости пересчитывает доплату и обновляет сведения в списке броней.

— Уведомление об истечении времени. Если до конца оплаченного или забронированного времени остаётся не более пятнадцати минут, в кабинете появляется заметное предупреждение с указанием номера места и оставшегося времени.

— Личный кабинет: профиль и приветствие. После входа пользователь попадает на страницу личного кабинета с обращением по имени, единой шапкой и навигацией; здесь же доступны карта мест, бронирование и история.

— Смена пароля. В кабинете пользователь может сменить пароль: вводит текущий и новый, система проверяет правильность старого пароля и требования к новому, после чего сохраняет изменения без выхода из сеанса.

— История поездок и сессий. В отдельном блоке кабинета выводится история прошлых и текущих парковок: место, период, статус, госномер и сведения об оплате — для просмотра, без редактирования архивных записей.

— Оплата и чек. Неоплаченную бронь можно оплатить в учебном демо-режиме без подключения банка; после оплаты формируется электронный чек с номером, суммой и периодом стоянки, который можно сохранить в виде документа.

— Панель администратора. Отдельный интерфейс для администратора: сводка по системе, управление пользователями и бронями, карта мест, аналитика загрузки и финансовые показатели. Доступ только у пользователей с административной ролью.

— Управление пользователями. Администратор просматривает список учётных записей, создаёт новых пользователей, меняет отображаемое имя и включает или отключает доступ к системе.

— Блокировка аккаунта с причиной. При отключении пользователя администратор может указать причину блокировки; при попытке входа заблокированный пользователь видит сообщение с этой причиной. Разблокировка выполняется повторным включением учётной записи.

— Просмотр и управление бронями. Пользователь отменяет и продлевает свои брони в кабинете. Администратор видит все брони в системе, может перенести бронь на другое место или продлить время стоянки.

— Ручной статус места. Администратор назначает месту статус «ремонт», «резерв» или «закрыто», в том числе одним действием на карте. Такое место не предлагается для бронирования и отображается с соответствующей пометкой, независимо от данных видеомониторинга.

— Аналитика по датам. По накопленным данным мониторинга формируются отчёты о загрузке парковки по часам и по дням за выбранный период; результаты выводятся в разделе аналитики админ-панели.

— Финансовая аналитика. За заданный период система показывает суммарную выручку, число оплат, средний чек и действующий тариф; эти же правила тарификации используются при расчёте стоимости и при оплате брони.

Аналоги и существующие решения:

— Системы платных парковок и муниципальные сервисы: ориентированы на оплату, зоны и справочную информацию; как правило не показывают статус каждого конкретного парковочного места в виде интерактивной карты;

— Паркинги с датчиками занятости: обеспечивают точность по месту, но требуют установки и обслуживания датчиков на каждом слоте;

— Универсальные картографические сервисы: Google Maps, Яндекс Карты, 2ГИС, Apple Maps — карта, POI, маршруты; нет ведомственного учёта статусов парковочных мест конкретной стоянки и связи с бронированием;

— Системы видеонаблюдения: дают поток с камеры, но без прикладной логики «слот — статус — карта» и без интеграции с сценариями бронирования и ручного управления доступностью;

Преимущества разрабатываемой системы:

— Предметная специализация: подсистема ориентирована на отображение доступности парковочных мест, а не на общую навигацию или оплату по зонам без привязки к слотам;

— Интеграция в одном контуре: мониторинг занятости, бронирование, ручные статусы и карта мест объединены в единую модель данных и единый интерфейс «Умная парковка»;

— Локальная актуальность: данные и сценарии привязаны к конкретной парковке и реальному набору мест, что позволяет демонстрировать работу модуля на практике;

— Прикладная полезность: пользователь видит, какие места свободны и где они расположены; администратор получает инструмент управления доступностью мест и исключения проблемных слотов из бронирования;

— Масштабируемость: архитектура (браузер — API — SQLite) позволяет расширять подсистему новыми источниками данных (камера, датчики), правилами статусов и аналитикой загрузки без переработки базовой логики отображения карты мест.

### 1.2 Математические (вербальные) модели

На диаграмме (рис. 1.1) представлена общая диаграмма деятельности подсистемы «Отображение свободных и занятых мест». Это схема последовательности действий пользователя и системы, а не схема базы данных. Процесс начинается со старта и загрузки исходных данных: результаты распознавания парковочных мест с камеры, сведения об активных бронированиях и ручные статусы мест, заданные администратором. Выполняется проверка корректности загрузки: при ошибке выводится сообщение и выполняется повторная попытка; при успехе объекты отображаются на кадре камеры, на карте мест и в счётчиках «свободно / занято / всего».

Далее пользователь выбирает сценарий работы:

— мониторинг с камеры — получение кадра, отправка на сервер, распознавание мест моделью YOLOv8-seg, классификация слотов, отрисовка контуров (зелёный — свободно, красный — занято), обновление счётчиков;

— карта мест — просмотр итоговых статусов слотов; выбор свободного места для бронирования с проверкой доступности;

— статусы мест — просмотр карты с учётом ручных ограничений; для администратора — установка статуса (закрыто, ремонт, резерв и т. п.) и сохранение в базе данных;

— сводка — агрегированные показатели по состояниям слотов и индикатор загрузки парковки.

Ветки объединяются на этапе обновления представления (карта мест, изображение с камеры и сводные показатели с учётом актуальных данных распознавания, броней и ручных статусов). В завершении проверяется, нужно ли продолжить работу: при положительном ответе процесс возвращается к выбору сценария; при отрицательном — завершение.

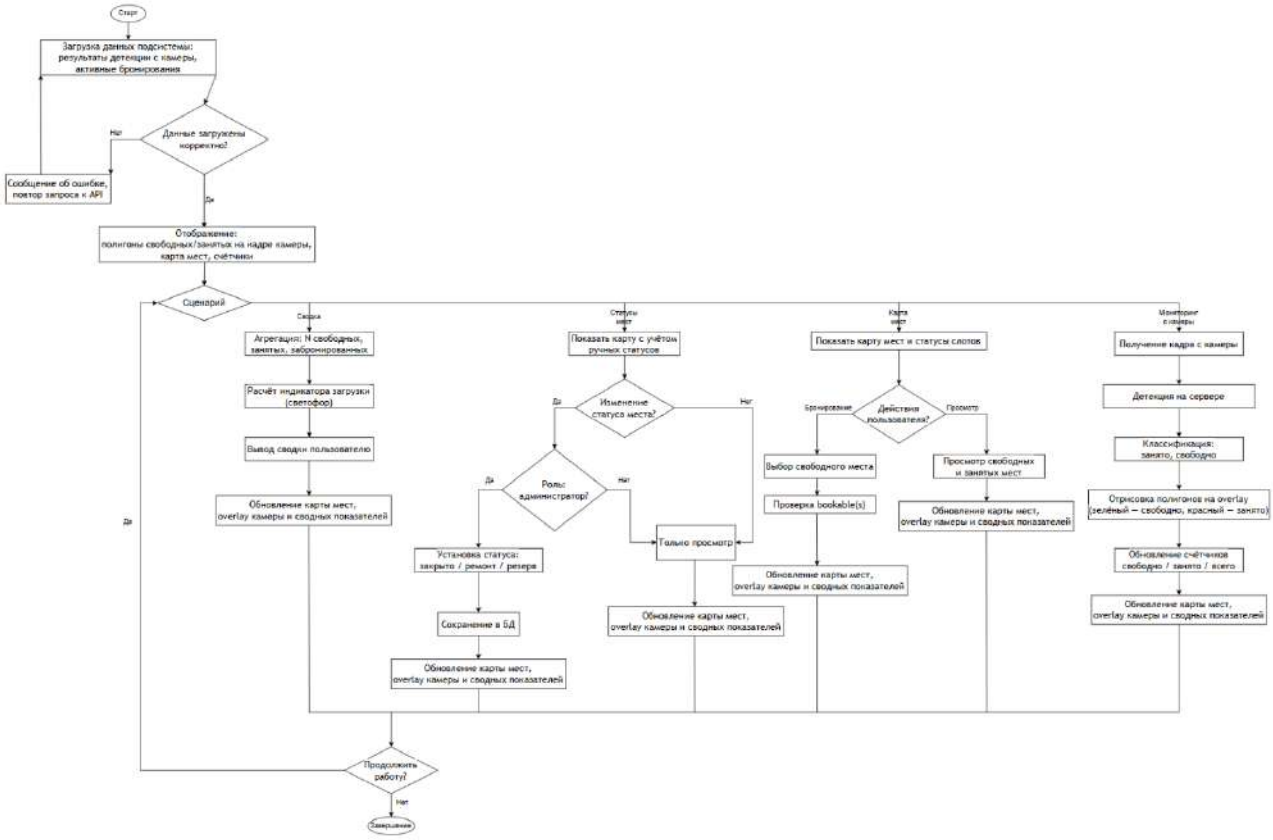



Рисунок 1.1 — Диаграмма деятельности подсистемы «Отображение свободных и занятых мест»

Модель 1. Итоговый статус парковочного места

Парковочное место s задаётся своим идентификатором. Используются три источника:

— detected(s) ∈ {free, occupied, unknown} — результат мониторинга по кадру (метки slot_free / slot_occupied после детекции YOLOv8-seg);

— booked(s) ∈ {0, 1} — признак активной брони (status = active, время окончания end_ts > t_now);

— override(s) ∈ {available, repair, reserved, closed, …} — ручной статус администратора.

Итоговый статус state(s), выдаваемый клиенту, определяется правилом приоритета:

$$state(s) = \begin{cases} \text{override(s)}, \text{если override(s)} \notin \{\text{available}, \text{free}, ""\}; \\ \text{booked}, \text{если } booked(s) = 1; \\ \text{detected(s)}, \text{иначе}. \end{cases} \quad (1.1)$$

Условие доступности места для бронирования bookable(s):

$$bookable(s) = \begin{cases} 1, \text{если override(s)} \in \{\text{available}, \text{free}, ""\} \wedge \text{booked(s)} = 0 \wedge \text{detected(s)} = \text{free}; \\ 0, \text{иначе} \end{cases} \quad (1.2)$$

Функция state(s) определяет цвет ячейки на карте мест и текст в подсказке (свободно, занято, забронировано, закрыто и т. д.).

Модель 2. Индикатор загрузки парковки («светофор»)

Пусть на парковке распознано N слотов мониторингом. Обозначим: N_total — число распознанных мест; N_free — число свободных по мониторингу; N_occ = N_total − N_free.

$$N_{total} = |N|,$$
$$N_{free} = |\{s \in N : detected(s) = free\}|,$$
$$N_occ = N_total - N_free.$$

Доля свободных мест:

$$p = \frac{N_{free}}{N_{total}} N_{total}, \quad \text{при } N_{total} > 0;$$
$$p = 0, \quad \text{при } N_{total} = 0. \quad (1.3)$$

Классификация уровня загрузки для визуализации задаётся порогами $p_1 = 0{,}1$ и $p_2 = 0{,}3$ (10 % и 30 % свободных мест):

$$level(p) = \begin{cases} green, & p > p_2; \\ yellow, & p_1 < p \le p_2; \\ red, & p \le p_1; \\ unknown, & N_{total} = 0. \end{cases} \quad (1.4)$$

Функция level(p) определяет цвет индикатора «светофора» и краткую сводку «свободно / всего» в интерфейсе мониторинга и личного кабинета.

Модель 3. Отображение слота на карте и кадре камеры

Множество слотов S = {$s_1$, $s_2$, …, $s_n$}. Для каждого формируются идентификатор, результат мониторинга, итоговый статус, ручной статус и признак доступности для брони.

$$\psi(s) = \begin{cases} 1, & \text{если } slot_{id(s)} \neq \text{пустое и слот присутствует в результате детекции;} \\ 0, & \text{иначе.} \end{cases} \quad (1.5)$$

На карте и на кадре показываются только слоты с ψ(s) = 1. Контур на камере — по мониторингу (зелёный/красный); ячейка карты — по state(s) по модели (1.1).

Модель 4. Сводные отчёты (агрегация)

По множеству слотов S с итоговыми статусами state(s) система формирует сводные показатели, например:

количество по итоговому состоянию: $N_\sigma = |\{s \in S : state(s) = \sigma\}|$, $\sigma \in \{free, occupied, booked, closed, repair, reserved, \dots\}$;

по результату детекции: $N_{\underset{free}{det}} = |\{s \in S : detected(s) = free\}|$, $N_{\underset{occ}{det}} = |\{s \in S : detected(s) = occupied\}|$;

число мест, недоступных для брони из-за ручного статуса: $N_{off} = |\{s \in S : override(s) \in \{repair, reserved, closed\}\}|$;

число мест с активной бронью: $N_{booked} = |\{s \in S : booked(s) = 1\}|$. (1.6)

Эти величины выводятся в блоке сводки и помогают пользователю выбрать место, а администратору — оценить загрузку и контролировать отключённые слоты.

# 2 Проектирование программного модуля

## 2.1 Перечень функциональности

Таблица 2.1 — Перечень функциональности подсистемы «Отображение свободных и занятых мест»

| Роль | Действие | Тип объекта | Объект | Дополнительные возможности | Сущность |
|---|---|---|---|---|---|
| Мониторинг с камеры и детекция | | | | | |
| Оператор / пользователь | Просматривает | Видеопоток | Кадр с камеры | Страница «Мониторинг парковки» (index.html): элемент video; поверх — canvas#overlay; режимы «Источник» (съёмка) и «Клиент» (поток с сервера) | (_last_slots в памяти сервера) |
| Оператор | Добавляет | Кадр | Изображение для детекции | Захват кадра с камеры или загрузка фото; POST /api/detect (multipart, x-api-key оператора); ответ: массив slots с полигонами points и метками slot_free / slot_occupied | детекция YOLOv8-seg |
| Пользователь | Просматривает | Canvas | Полигоны слотов на overlay | Функция drawAutoSlots (static/app.js): зелёная заливка — slot_free, красная — slot_occupied; координаты points нормализованы относительно кадра | static/app.js |
| Пользователь | Просматривает | Панель | Счётчики занятости | Блок stat-grid: «Свободно», «Занято», «Всего мест»; обновление по результату последней детекции | counts (вычисление на клиенте) |
| Пользователь | Просматривает | Данные API | Состояние мониторинга | GET /api/parking/state (x-api-key клиента): ts, slots, counts; read-only снимок последней детекции | (_last_slots, _last_ts) |
| Пользователь | Просматривает | Индикатор | Светофор загрузки | GET /api/parking/traffic: level (green / yellow / red), pct_free, free, total, occupied; пороги: $p > 0{,}3$ — green; $0{,}1 < p \le 0{,}3$ — yellow; $p \le 0{,}1$ — red | traffic (расчёт _traffic_from_counts) |
| Карта мест (свободно занято) | | | | | |
| Пользователь | Просматривает | Карта | Карта мест | Личный кабинет (cabinet.html): GET /api/parking/slots-map; сетка ячеек slot-map__cell; цвет по state(s): free, occupied, booked, closed и др. | bookings, slot_overrides, _last_slots |
| Пользователь | Просматривает | Подсказка | Парковочное место | Атрибут title ячейки: slot_id, state, override_note; поля ответа API: id, detected, state, override, override_note, bookable | (_parking_slots_map_payload) |
| Пользователь | Выбирает | Карта | Свободное место | Клик по ячейке с bookable=true; ParkingFeatures.renderSlotMap → onSelect; slot_id подставляется в форму «Новая бронь» (связанный сценарий бронирования) | static/pe-features.js, cabinet.js |

Продолжение таблицы 2.1

| Роль | Действие | Тип объекта | Объект | Дополнительные возможности | Сущность |
|---|---|---|---|---|---|
| Пользователь | Просматривает | Данные API | Итоговый статус места | Сервер объединяет detected, активную бронь и override; правило приоритета — модель (1.1); bookable — модель (1.2) | bookings, slot_overrides |
| Ручные статусы и администрирование карты | | | | | |
| Администратор | Просматривает | Карта | Карта мест (админ) | admin.html: GET /api/parking/slots-map; ParkingFeatures.renderAdminSlotMap — клик по любой ячейке | slot_overrides, bookings |
| Администратор | Просматривает | Список | Ручные статусы | GET /api/admin/slot-overrides: список записей slot_id, status, note, updated_ts | slot_overrides |
| Администратор | Изменяет | Существующий | Статус места | PUT /api/admin/slot-overrides/{slot_id}: status ∈ {available, repair, reserved, closed}, note; при клике на карте — переключение closed ↔ available | slot_overrides |
| Администратор | Изменяет | Существующий | Доступность для брони | После override статус closed/repair/reserved имеет приоритет над детектом; место отображается на карте с соответствующим state и bookable=false | slot_overrides, bookings |
| Администратор | Ищет | Форма | Место по госномеру | GET /api/admin/find-by-plate?plate=...; поиск среди active-броней; возврат slot_id, live_state, live_detected; подсветка найденного слота на адмін-карте | bookings.plate |
| Сводка загрузки парковки | | | | | |
| Пользователь | Просматривает | Панель | Сводка по слотам | В ответе /api/parking/slots-map: объект counts (total, free, occupied) и traffic; отображение в кабинете и на странице мониторинга | counts, traffic |
| Пользователь | Просматривает | Индикатор | Уровень загрузки | ParkingFeatures.renderTraffic: визуализация green / yellow / red по pct_free; используется в monitorTraffic и в кабинете | static/pe-features.js |
| Пользователь | Просматривает | Данные API | Актуальность данных | Поле ts — время последней детекции; при отсутствии данных traffic.level = unknown, counts нулевые | _last_ts |

## 2.2 Сценарии взаимодействия

Сценарии описывают, как ведёт себя подсистема «Отображение свободных и занятых мест» в приложении «Умная парковка»: краткий вариант задаёт смысл, подробный — пошаговый основной путь и ответы системы.

Пользователь после входа работает в личном кабинете: просматривает карту мест и список своих броней, рассчитывает стоимость по длительности, оформляет и оплачивает парковку, продлевает или отменяет бронь, получает предупреждение перед окончанием времени и сохраняет чек. Администратор использует отдельную панель: управляет учётными записями и бронями, назначает ручные статусы мест, анализирует загрузку парковки и финансовые показатели. Ниже каждый сценарий соответствует одной из пятнадцати ключевых функций подсистемы.

Сценарий А. Отмена брони в один клик

Пользователь открывает личный кабинет и переходит к списку активных броней. Напротив нужной записи нажимает кнопку «Отменить». Система запрашивает подтверждение действия (при необходимости) и снимает бронь. Запись в списке получает статус «отменена»; место снова может отображаться как доступное на карте. Система показывает краткое сообщение об успешной отмене.

Альтернатива: бронь уже завершена или отменена — кнопка недоступна; при ошибке связи пользователь повторяет действие или обновляет страницу.

Сценарий Б. Расчёт стоимости по тарифу

Пользователь в кабинете открывает форму новой брони или изменяет поле «Длительность, ч». При каждом изменении длительности система пересчитывает стоимость по действующему почасовому тарифу. Рядом с формой отображается строка вида «≈ N ₽ за выбранное время». Пользователь видит сумму до нажатия «Забронировать» и может скорректировать длительность.

Альтернатива: тариф временно недоступен — показывается сообщение об ошибке; после восстановления связи расчёт выполняется снова.

Сценарий В. Продление времени парковки

Пользователь в списке броней находит активную запись с оплаченным или подтверждённым временем. Нажимает команду продления (например, «+1 ч»). Система увеличивает время окончания стоянки и при необходимости показывает сумму доплаты. Список броней и предупреждения об истечении времени обновляются.

Альтернатива: бронь уже истекла — продление недоступно; пользователь оформляет новую бронь.

Сценарий Г. Предупреждение за пятнадцать минут до конца

Пользователь находится в личном кабинете при активной брони. Система периодически проверяет оставшееся время по каждой активной записи. Если до окончания остаётся не более пятнадцати минут, в верхней части кабинета появляется жёлтый баннер с номером места и примерным числом минут. Баннер исчезает после продления времени, отмены брони или истечения срока.

Альтернатива: несколько броней заканчиваются одновременно — выводится несколько предупреждений подряд.

Сценарий Д. Вход в личный кабинет и приветствие

Пользователь выполняет вход в систему под своей учётной записью. Открывает страницу «Личный кабинет» через меню приложения. Система подставляет имя пользователя в приветствие и отображает единую шапку с навигацией. На странице доступны карта мест, форма бронирования, список броней, история и настройки профиля.

Альтернатива: сеанс истёк — пользователь перенаправляется на страницу входа.

Сценарий Е. Смена пароля

Пользователь в кабинете открывает блок смены пароля. Вводит текущий пароль и новый пароль дважды (при наличии поля подтверждения). Нажимает «Сохранить». Система проверяет правильность текущего пароля и соблюдение требований к новому. При успехе выводится сообщение; пользователь остаётся в кабинете под той же учётной записью.

Альтернатива: неверный текущий пароль или слишком простой новый — сообщение об ошибке, пароль не меняется.

Сценарий Ж. Просмотр истории поездок

Пользователь прокручивает кабинет до блока «История сессий» (или «История поездок»). Система загружает список прошлых и текущих парковок пользователя (и при семейном доступе — записей семьи). В таблице отображаются место, период, статус, госномер и сведения об оплате. Пользователь просматривает архив без возможности редактирования старых записей.

Альтернатива: история пуста — выводится текст «Нет данных»; при сбое загрузки — предложение обновить страницу.

Сценарий З. Оплата брони и получение чека

Пользователь в списке броней выбирает активную неоплаченную запись. Нажимает «Оплатить» (демо-режим без реального банка). Система фиксирует оплату, присваивает номер чека и показывает сумму. Появляется кнопка «Чек» или чек скачивается автоматически в виде документа. Пользователь сохраняет файл с номером чека, суммой и периодом стоянки.

Альтернатива: бронь уже оплачена — предлагается повторно скачать чек; при ошибке — повтор оплаты или обращение в поддержку.

Сценарий И. Работа в панели администратора

Администратор входит под учётной записью с административной ролью. Открывает страницу «Администрирование». Система загружает сводку, списки пользователей и броней, карту мест, блоки аналитики и финансов. Обычный пользователь не видит эту страницу или получает отказ в доступе при попытке открыть её. Администратор переходит между разделами панели для выполнения служебных операций.

Альтернатива: сеанс администратора истёк — требуется повторный вход.

Сценарий К. Создание и отключение пользователей

Администратор открывает таблицу пользователей в админ-панели. Для нового сотрудника или клиента заполняет форму: логин, пароль, имя, роль — и сохраняет. Новая учётная запись появляется в списке и может войти в систему. Для существующего пользователя администратор нажимает «Вкл/Выкл» или меняет отображаемое имя. Список обновляется после каждого изменения.

Альтернатива: логин уже занят — сообщение об ошибке; администратор не может отключить собственную учётную запись.

Сценарий Л. Блокировка аккаунта с указанием причины

Администратор отключает пользователя в таблице учётных записей. Система предлагает ввести причину блокировки (необязательный, но рекомендуемый текст). Причина сохраняется вместе с признаком «неактивен». Заблокированный пользователь при входе видит сообщение о блокировке и текст причины. Разблокировка выполняется повторным включением учётной записи и очисткой причины.

Альтернатива: пользователь пытается войти до сохранения блокировки — после обновления статуса вход блокируется.

Сценарий М. Управление бронями администратором

Администратор открывает список всех броней в системе. Просматривает место, пользователя, период и статус каждой записи. При необходимости переносит бронь на другое место, указывая новый номер слота. Может продлить время стоянки для выбранной брони. После изменения список и карта мест обновляются; пользователь видит актуальные данные в своём кабинете.

Альтернатива: целевое место занято или закрыто — система сообщает об ошибке; администратор выбирает другое место.

Сценарий Н. Назначение ручного статуса места

Администратор открывает карту мест в админ-панели или форму ручного статуса. Выбирает место и задаёт статус «ремонт», «резерв» или «закрыто», при необходимости — комментарий. Система сохраняет статус в базе данных; на карте место отображается с соответствующей пометкой. Место перестаёт предлагаться для бронирования независимо от данных видеомониторинга. Снятие ограничения выполняется возвратом статуса «доступно» или повторным действием на админ-карте.

Альтернатива: попытка обычного пользователя изменить статус — операция отклоняется.

Сценарий О. Аналитика загрузки парковки

Администратор открывает раздел «Аналитика» в админ-панели. Указывает период анализа (например, последние семь дней) или использует значение по умолчанию. Система агрегирует накопленные данные мониторинга: среднюю и пиковую занятость по дням, распределение по часам. Результаты выводятся в виде карточек и таблицы «По дням». Администратор использует отчёт для оценки пиковых нагрузок и планирования.

Альтернатива: мониторинг ещё не передавал данные — сообщение «Нет наблюдений»; отчёт появится после работы камеры.

Сценарий П. Финансовая аналитика

Администратор открывает раздел «Финансы» в админ-панели. Система формирует сводку за выбранный период (например, тридцать дней). Отображаются суммарная выручка, число оплат, средний чек и действующий почасовой тариф. Показатели согласованы с расчётом стоимости в кабинете и с суммами в чеках пользователей. Администратор использует сводку для контроля доходности учебного прототипа парковки.

Альтернатива: за период не было оплат — выручка равна нулю, тариф всё равно отображается для справки.

## 2.3 Прототипирование интерфейса взаимодействия с пользователем (графического интерфейса).

На рисунке 2.1 представлена страница «Мониторинг парковки» — основная рабочая область подсистемы «Отображение свободных и занятых мест». Слева — общая боковая навигация приложения «Умная парковка»: главная, мониторинг, кабинет, админ-панель. В основной области — заголовок страницы, переключатель режимов «Источник» и «Клиент», блок видеопотока с наложением контуров парковочных мест (зелёный — свободно, красный — занято), справа — панель мониторинга: индикатор «светофор», счётчики «Свободно / Занято / Всего», панель действий (камера, снять фото, загрузить, сводка), блок со списком мест. Для индивидуального задания выделены зоны наложения на кадр, счётчиков и светофора; остальные элементы панели действий относятся к общему контуру мониторинга команды.

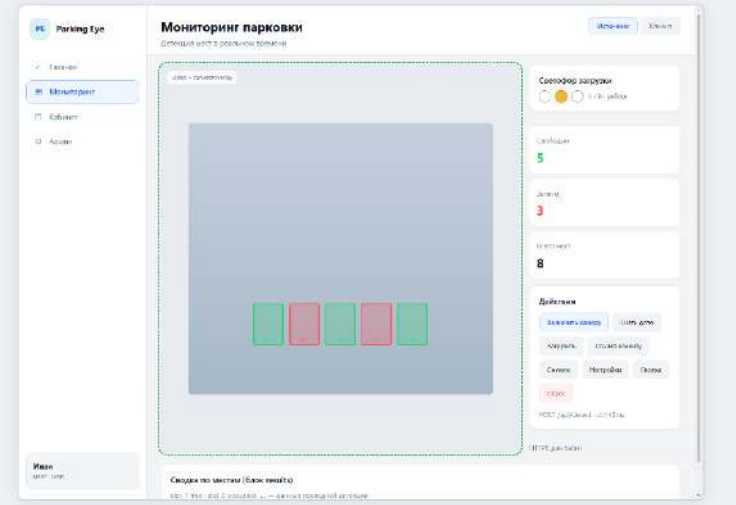



Рисунок 2.1 – Рисунок макета страницы «Мониторинг парковки» (полный экран)

На рисунке 2.2 показана страница «Личный кабинет» с блоком «Карта мест». В центре — сетка ячеек: каждая соответствует номеру места и окрашена по итоговому состоянию (свободно, занято, забронировано, закрыто). Над картой — подсказка «Нажмите на свободное место для брони»; при наведении отображаются номер места, состояние и примечание администратора. Свободные доступные места реагируют на щелчок и подставляют номер в форму «Новая бронь» ниже. Рядом может отображаться индикатор загрузки парковки. Снимок иллюстрирует просмотр свободных и занятых мест и выбор места пользователем.

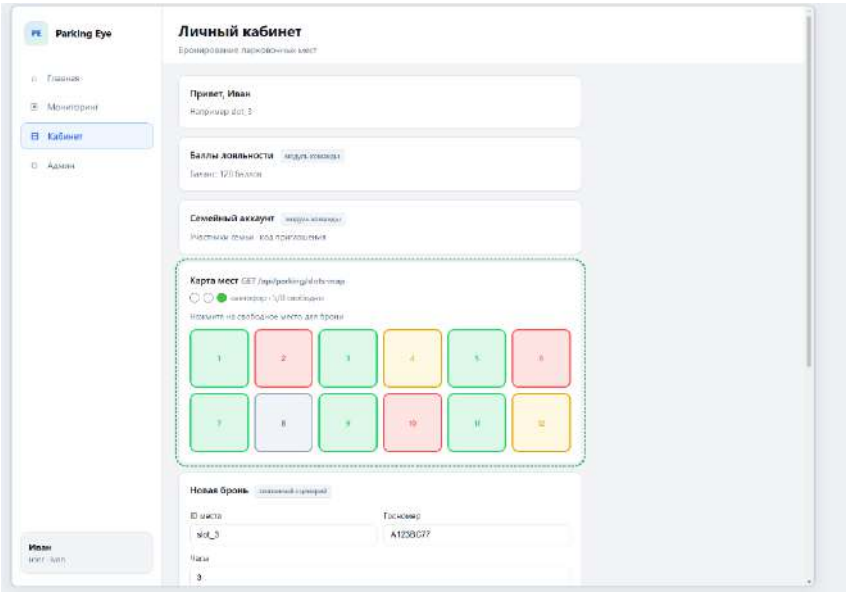



Рисунок 2.2 – Рисунок макета страницы «Личный кабинет»: карта мест и цветовая индикация статусов

На рисунке 2.3 представлена страница «Администрирование» — блок «Статусы мест». Отображается карта мест для администратора: щелчок по ячейке переключает ручной статус «закрыто» и «доступно». Ниже — список текущих ручных статусов с номером места, состоянием и комментарием. Выше на странице — форма «Поиск по госномеру» с подсветкой найденного места на карте. Для учётной записи администратора доступно управление доступностью мест; блоки пользователей, финансов и чата поддержки относятся к смежным модулям команды. Снимок иллюстрирует администрирование отображаемых статусов парковочных мест.

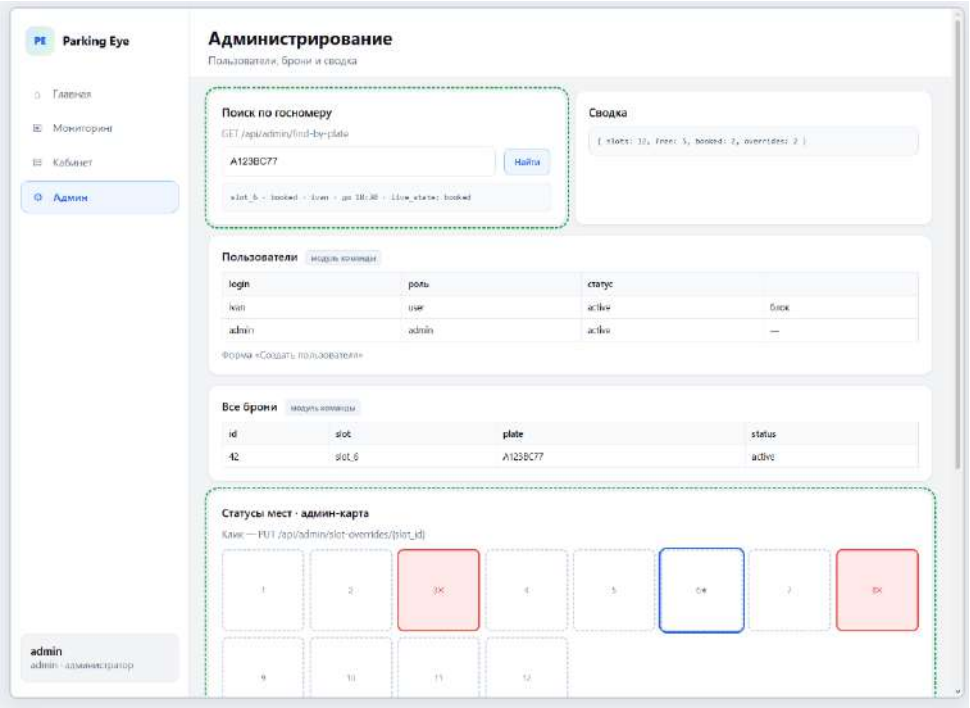



Рисунок 2.3 – Рисунок макета страницы «Администрирование»: админ-карта мест и ручные статусы

На рисунке 2.4 представлена страница «Главная» после входа пользователя. Слева — боковая навигация приложения. В основной области — приветствие, карточки перехода к мониторингу и кабинету, блок «Парковка сейчас» с индикатором «светофор», блок ближайшей брони и быстрые ссылки на режимы камеры и админ-панель. Страница выполняет роль стартового экрана: пользователь оценивает загрузку парковки и переходит к мониторингу с камеры или к карте мест в кабинете.

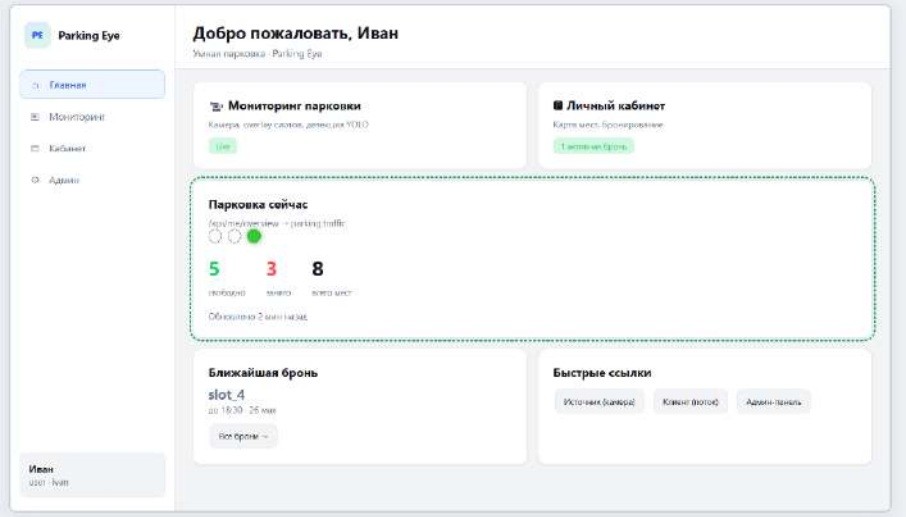



Рисунок 2.4 – Рисунок макета главного экрана «Parking Eye» (dashboard.html, полный экран)

## 2.4 Составление индивидуального плана работы

Индивидуальный план ориентирован на подсистему «Отображение свободных и занятых мест» в составе единого проекта ИС «Умная парковка». Календарные рамки: 15 недель (09.02.2026 — 20.05.2026); общий репозиторий совпадает с командным планом. Ниже по каждой неделе выделены задачи и результаты в объёме индивидуального задания.

Недели 1–2. Сбор аналогов: системы умных парковок, видеоаналитики занятости мест, приложения бронирования парковок; таблица сравнения (плюсы и минусы, способ хранения данных). Фиксация предметной области: парковочное место с итоговым состоянием; распознавание свободных и занятых мест с камеры; ручной статус администратора; индикатор загрузки («светофор»). Границы с модулями команды (вход в систему, бронирование, оплата, семейный аккаунт, аналитика; смежные задачи напарника — тарифы, электронные чеки и т. п.).

Результат: обоснована актуальность подсистемы; черновой список требований; глоссарий и границы задачи согласованы с командой.

Неделя 3. Встраивание в общую архитектуру: страница мониторинга с видео и наложением на кадр, карта мест в личном кабинете и админ-панели; согласование с общим оформлением и навигацией приложения.

Результат: схема экранов (wireframe) — рис. 2.1–2.4.

Неделя 4. Серверная часть и обмен данными: приём кадра с камеры, выдача карты мест и сводки загрузки; описание правил объединения данных мониторинга, броней и ручных статусов.

Результат: зафиксирован перечень операций сервера и место подсистемы в цепочке «интерфейс — сервер — база данных».

Неделя 5. Проектирование хранения ручных статусов мест и связи с бронированиями; правила приоритета итогового состояния и доступности места для брони — модели (1.1)–(1.2).

Результат: фрагмент модели данных (ER-диаграмма) или описание структуры таблиц.

Неделя 6. Реализация структуры базы данных и тестовых данных: несколько мест, ручные статусы «закрыто» и «ремонт», активные брони; проверка правил объединения состояний.

Результат: в базе данных созданы объекты, достаточные для интерфейса и серверной логики.

Неделя 7. Макеты: наложение на кадр камеры, счётчики, карта мест, индикатор «светофор», админ-карта; согласование с командой.

Результат: wireframes или скриншоты прототипов.

Неделя 8. Черновая реализация интерфейса: отрисовка мест на кадре и на карте, первичная загрузка актуальных данных с сервера.

Результат: в общем интерфейсе видны зоны ответственности подсистемы.

Неделя 9. Карта мест в кабинете, индикатор загрузки на мониторинге и главной странице; отображение результата распознавания и итогового состояния каждого места.

Результат: карта и сводка формируются из реальных данных сервера.

Неделя 10. Мониторинг с камеры: отправка кадра на сервер, отрисовка контуров мест, обновление счётчиков; стабильные номера мест при повторных снимках.

Результат: сценарий мониторинга с камеры работает от начала до конца.

Неделя 11. Администрирование мест: ручная установка статуса по карте, список ручных ограничений, поиск активной брони по госномеру с подсветкой места на карте.

Результат: полный цикл ручного управления отображаемым статусом места.

Недели 12–13. Сквозные испытания: от мониторинга до карты мест и бронирования; проверка отказа в доступе обычному пользователю к админ-функциям, недоступности занятого места, порогов индикатора «светофор». Оформление плана испытаний, скриншоты, исправление ошибок.

Результат: согласован жизненный цикл отображения состояния мест; сценарии тестирования зафиксированы.

Недели 14–15. Улучшение карты мест и серверной логики; понятные сообщения об ошибках (сеть, отсутствие данных с камеры). Подготовка пояснительной записки: требования, модели, описание взаимодействия, диаграммы, раздел

тестирования, рис. 1.1 и 2.1–2.4. Сценарий демонстрации: камера → кабинет → карта → админ-управление местами → индикатор загрузки; репетиция защиты.

Результат: стабильный интерфейс; пояснительная записка и материалы защиты закрывают подсистему по плану.

Таблица 2.2– Календарный план работ группы разработчиков по созданию ИС «Умная парковка» (15 недель)

| № | 09.02 | 16.02 | 23.02 | 02.03 | 09.03 | 16.03 | 23.03 | 30.03 | 06.04 | 13.04 | 30.02 | 27.04 | 04.05 | 11.05 | 20.05 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | ■ | | | | | | | | | | | | | | |
| 2 | | ■ | | | | | | | | | | | | | |
| 3 | | | ■ | | | | | | | | | | | | |
| 4 | | | | ■ | | | | | | | | | | | |
| 5 | | | | | ■ | | | | | | | | | | |
| 6 | | | | | | ■ | | | | | | | | | |
| 7 | | | | | | | ■ | | | | | | | | |
| 8 | | | | | | | | ■ | | | | | | | |
| 9 | | | | | | | | | ■ | | | | | | |
| 10 | | | | | | | | | | ■ | | | | | |
| 11 | | | | | | | | | | | ■ | | | | |
| 12 | | | | | | | | | | | | ■ | | | |
| 13 | | | | | | | | | | | | | ■ | | |
| 14 | | | | | | | | | | | | | | ■ | |
| 15 | | | | | | | | | | | | | | | ■ |

## 3. Разработка документации программного модуля

### 3.1 Диаграмма классов

При проектировании индивидуальной подсистемы «Отображение свободных и занятых мест» принята трёхслойная организация: пользовательский интерфейс, бизнес-логика и данные. Такое разделение обеспечивает независимость отображения (карта мест, наложение на кадр камеры, индикатор «светофор») от правил объединения статусов и от структуры базы данных. Ниже по каждому слою приведена диаграмма классов и описание связей между классами.

На рисунке 3.1 представлена диаграмма классов слоя пользовательского интерфейса подсистемы. На ней отражены экранные компоненты, через которые пользователь и администратор выполняют сценарии индивидуального задания: мониторинг с наложением контуров на кадр (свободно / занято), просмотр карты мест и выбор свободного места, управление ручными статусами и поиск автомобиля по госномеру. Все обращения к серверу сведены к единому шлюзу — классу ParkingApiAdapter, что отделяет разметку и отрисовку от сетевого обмена данными.

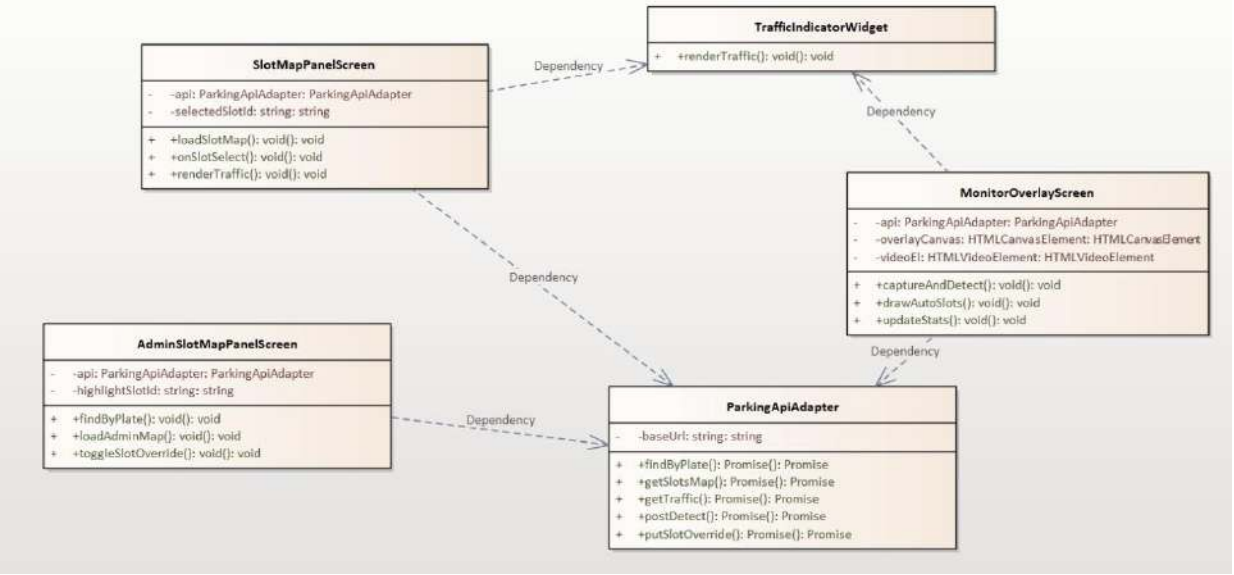



Рисунок 3.1 – Диаграмма классов слоя пользовательского интерфейса подсистемы «Отображение свободных и занятых мест»

MonitorOverlayScreen — страница «Мониторинг парковки»: видеопоток, слой наложения на кадр, контуры мест, счётчики «свободно / занято», отправка снимка на распознавание. Зелёная и красная подсветка — «свободно» и «занято».

SlotMapPanelScreen — блок «Карта мест» в кабинете: загрузка данных, сетка ячеек по итоговому состоянию, выбор места для брони, индикатор загрузки.

AdminSlotMapPanelScreen — админ-карта: ручной статус по щелчку, поиск по госномеру, подсветка найденного места.

TrafficIndicatorWidget — «светофор»: уровни «зелёный», «жёлтый», «красный».

ParkingApiAdapter — единый шлюз запросов карты мест, сводки загрузки, распознавания кадра, ручных статусов и поиска по номеру.

Связи между классами слоя интерфейса. MonitorOverlayScreen через ParkingApiAdapter отправляет кадр и получает результат; рядом использует TrafficIndicatorWidget. SlotMapPanelScreen и AdminSlotMapPanelScreen тоже зависят от ParkingApiAdapter (карта для пользователя и для администратора). TrafficIndicatorWidget вызывают мониторинг, кабинет и главная страница. Все экраны зависят от адаптера и не обращаются к базе напрямую и не вычисляют итоговое состояние места.

Экранные компоненты не вычисляют итоговый статус — это делает сервер. MonitorOverlayScreen показывает результат распознавания; карты в кабинете и админке — уже объединённое состояние. При изменении формата обмена правки в основном в ParkingApiAdapter и серверной части.

На рисунке 3.2 представлена диаграмма классов слоя бизнес-логики подсистемы «Отображение свободных и занятых мест». На одной схеме показаны координатор сценариев, специализированные сервисы, класс пользователя и классы доступа к данным, с которыми сервисы работают при формировании карты мест и сводки загрузки. Диаграмма отражает правила объединения распознавания с камеры, активных бронирований и ручных статусов (модели 1.1–1.2), расчёт индикатора «светофор» (модель 1.4), обработку кадра и административные операции.

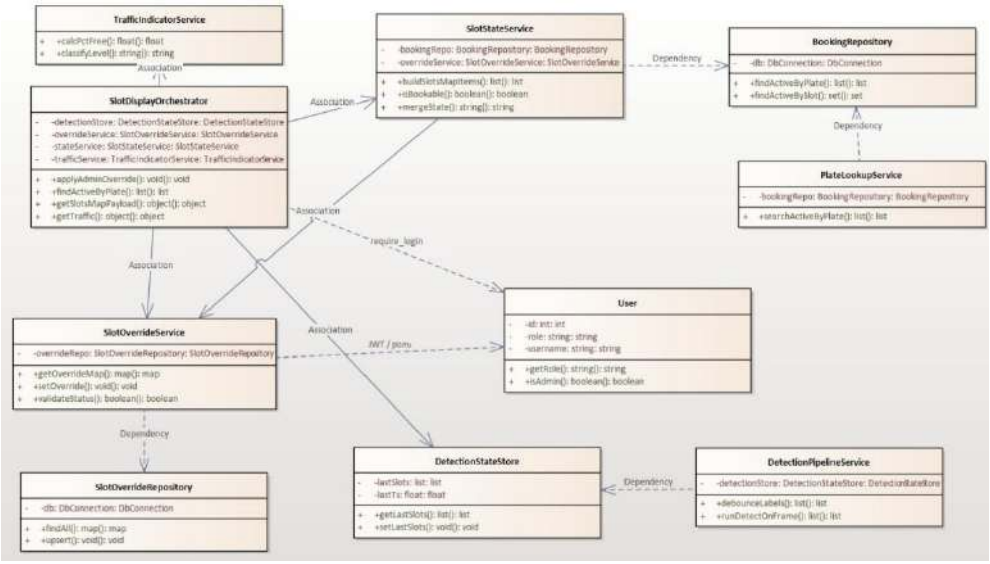



Рисунок 3.2 – Диаграмма классов слоя бизнес-логики подсистемы «Отображение свободных и занятых мест»

SlotDisplayOrchestrator — центральный координатор. Принимает запросы от интерфейса и направляет их в нужные сервисы: формирует данные карты мест, возвращает сводку загрузки, передаёт команду на ручной статус, инициирует поиск по госномеру. Связан с сервисами состояния, загрузки, ручных статусов и с хранилищем последней детекции.

SlotStateService — ядро правил отображения. Для каждого места объединяет результат мониторинга («свободно» / «занято»), активную бронь и ручной статус; вычисляет итоговое состояние (модель 1.1), признак «можно бронировать» (модель 1.2), список для карты. Использует репозиторий бронирований и сервис ручных статусов.

TrafficIndicatorService — общая загрузка парковки: доля свободных мест и уровень «зелёный» / «жёлтый» / «красный» по порогам 30 % и 10 % (модель 1.4).

DetectionPipelineService — кадр с камеры: распознавание YOLOv8-seg, устойчивые номера слотов, сглаживание меток, запись в хранилище последней детекции. Брони не меняет.

SlotOverrideService — ручные статусы («доступно», «ремонт», «резерв», «закрыто»): проверка допустимости, сохранение и снятие ограничения; только для администратора. Работает через репозиторий ручных статусов.

PlateLookupService — поиск активных броней по нормализованному госномеру для администратора, с учётом текущего состояния места на карте. Использует репозиторий бронирований.

User — учётная запись и роль (пользователь / администратор). Без входа недоступны карта и сводка; без роли администратора — ручной статус и поиск по номеру.

BookingRepository — выборка активных броней по номеру места и по госномеру из базы данных.

SlotOverrideRepository — чтение всех ручных ограничений и сохранение записи по номеру места.

DetectionStateStore — последний результат распознавания в памяти сервера (список мест и время). Нужен для карты и «светофора» до следующего кадра.

SlotDisplayOrchestrator ассоциирован с SlotStateService, TrafficIndicatorService, SlotOverrideService и DetectionStateStore: карта мест → SlotStateService; сводка → TrafficIndicatorService; смена статуса → SlotOverrideService; актуальный список с камеры → DetectionStateStore.

SlotStateService зависит от BookingRepository (активные брони) и связан с SlotOverrideService (ручные статусы). Здесь выполняются слияние состояний (mergeState), проверка доступности (isBookable) и подготовка списка для карты (buildSlotsMapItems).

SlotOverrideService зависит от SlotOverrideRepository и User (только администратор).

PlateLookupService зависит от BookingRepository; результат сопоставляется с данными карты при подсветке места.

DetectionPipelineService зависит от DetectionStateStore: после обработки кадра обновляется снимок, который читают координатор и SlotStateService.

BookingRepository и SlotOverrideRepository на рис. 3.2 показаны в контуре бизнес-логики, потому что сервисы обращаются к ним при каждом построении карты; физически они работают с базой данных (подробнее на рис. 3.3).

SlotDisplayOrchestrator зависит от User (обязательный вход); административные операции дополнительно проверяют роль администратора.

На рисунке 3.3 представлена диаграмма классов слоя доступа к данным подсистемы «Отображение свободных и занятых мест». На схеме четыре класса: единое подключение к базе данных, два репозитория постоянных записей и оперативное хранилище последнего результата распознавания с камеры.

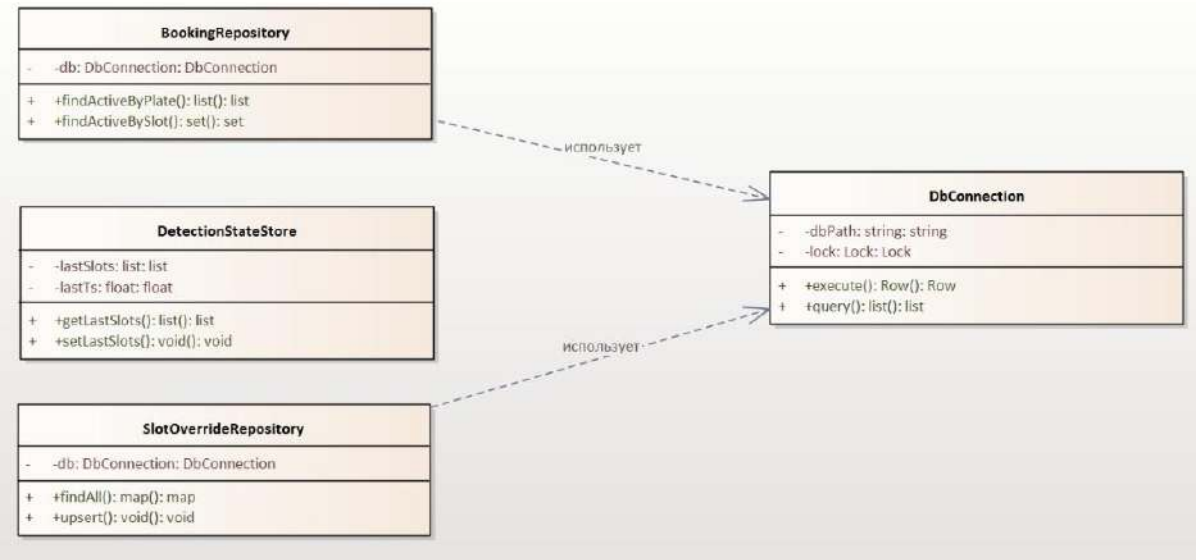



Рисунок 3.3 – Диаграмма классов слоя доступа к данным подсистемы «Отображение свободных и занятых мест»

DbConnection — общий доступ к файлу базы данных приложения. Хранит путь к файлу базы и механизм блокировки при одновременных запросах от нескольких пользователей. Предоставляет две операции: выполнение команд изменения данных (вставка, обновление, удаление) и выборка данных списком записей. Все обращения к таблицам бронирований и ручных статусов проходят через этот класс.

BookingRepository — работа с таблицей бронирований. Содержит ссылку на DbConnection. Метод выборки по номеру места возвращает множество мест с

активной бронью; используется при построении карты (отметка «забронировано»). Метод выборки по госномеру возвращает список активных броней с указанным номером автомобиля; используется при поиске администратором.

SlotOverrideRepository — работа с таблицей ручных статусов мест. Также использует DbConnection. Метод получения всех записей возвращает текущую карту ограничений (номер места, статус, комментарий, время изменения). Метод сохранения добавляет новую запись или обновляет существующую по номеру места — основа переключения «закрыто / доступно» и назначения «ремонт» или «резерв» на адмін-карте.

DetectionStateStore — оперативное хранилище в памяти сервера, не связанное с файлом базы данных. Хранит список мест последнего распознавания с камеры и время его получения. Методы чтения и записи позволяют сохранить новый снимок после обработки кадра и прочитать актуальный результат до следующего распознавания. Данные здесь меняются часто; для архива загрузки парковки в проекте дополнительно используется отдельная таблица наблюдений (на данной диаграмме не показана).

BookingRepository и SlotOverrideRepository связаны с DbConnection зависимостью «использует» (пунктирная стрелка): репозитории делегируют соединение и выполнение запросов классу DbConnection. DetectionStateStore не связан с DbConnection — это разделение постоянных данных (брони, ручные статусы) и оперативного снимка с камеры.

Со стороны бизнес-логики (рис. 3.2) к BookingRepository и SlotOverrideRepository обращаются SlotStateService, SlotOverrideService и PlateLookupService; к DetectionStateStore — SlotStateService, SlotDisplayOrchestrator и DetectionPipelineService. Слой данных поставляет три источника: распознавание (DetectionStateStore), брони (BookingRepository) и ручные ограничения (SlotOverrideRepository); SlotStateService объединяет их по моделям (1.1)–(1.2).

Разделение BookingRepository и DetectionStateStore отражает два вида занятости: «занято по камере» и «забронировано по записи в базе», плюс ручной статус из SlotOverrideRepository. Контуры на кадре и итоговые цвета карты могут согласованно различаться (место свободно по камере, но закрыто администратором или забронировано пользователем).

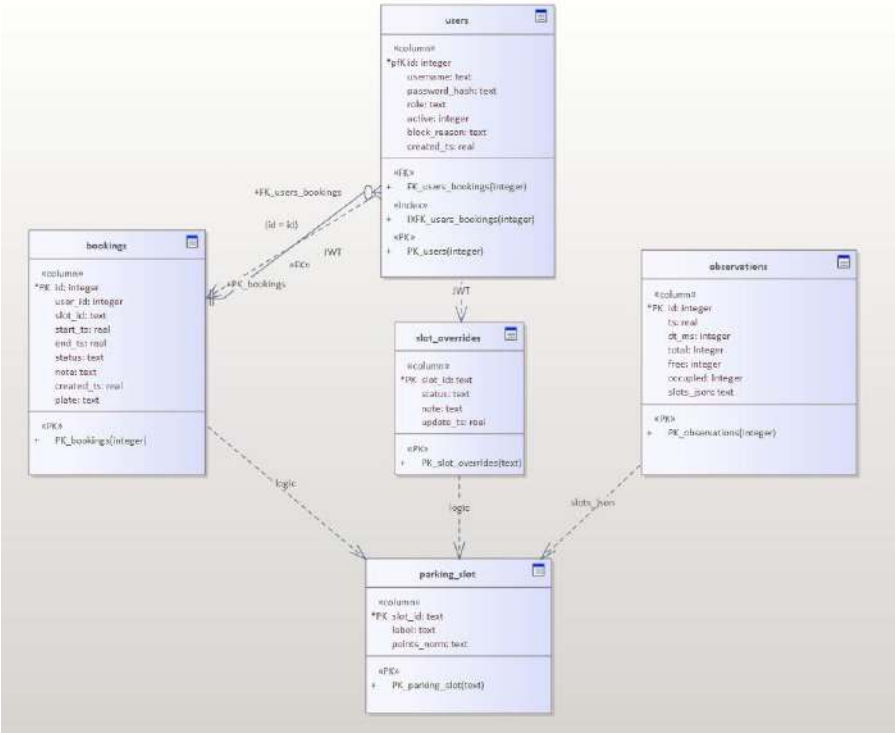



Рисунок 3.4 – Диаграмма базы данных

Сущность users (пользователи). Содержит учётные записи и роли (user, admin). Поля: id — первичный ключ; username и password_hash — логин и хеш пароля; display_name — отображаемое имя; role — роль в системе; active — признак активности; block_reason — причина блокировки; created_ts — время создания записи. Сущность используется для проверки входа и прав доступа перед выдачей карты мест и изменением ручных статусов; в slot_overrides столбца user_id нет.

Сущность bookings (бронирования мест). Фиксирует резервирование парковочного места по идентификатору slot_id на интервал времени. Поля: id — первичный ключ; user_id — владелец брони (ссылка на users); slot_id — строковый идентификатор места (например, slot_12); start_ts и end_ts — начало и конец интервала (Unix-время, REAL); status — состояние записи (active, cancelled, completed); note — примечание; created_ts — момент создания; plate — нормализованный госномер для поиска администратором. Составной индекс idx_bookings_slot_active ускоряет выборку активных броней по slot_id при построении карты мест и поиске по номеру. На карте пометка «забронировано» появляется, если для slot_id есть строка со status = active и end_ts позже текущего момента (модель 1.2, раздел 1.2).

Сущность slot_overrides (ручные статусы мест). Хранит принудительные статусы, которые администратор назначает месту: available, repair, reserved, closed и др. Поля: slot_id — первичный ключ (TEXT); status — назначенный статус; note — комментарий; updated_ts — время последнего изменения. Отдельной таблицы «слотов» в базе нет: идентификатор совпадает с id из конфигурации и результата детекции.

Статус override имеет наивысший приоритет при вычислении итогового состояния места (модель 1.1, раздел 1.2).

Сущность observations (архив наблюдений мониторинга). Фиксирует агрегаты каждого кадра детекции: id — первичный ключ; ts — время кадра; dt_ms — длительность распознавания; total, free, occupied — счётчики мест; slots_json — JSON-текст со списком слотов и их состоянием на момент кадра. Запись выполняется асинхронно после обработки кадра камерой. Для live-карты мест и overlay используется снимок в памяти сервера, а не последняя строка observations; таблица нужна для истории командного проекта и аналитики, но не является обязательным источником индивидуальной подсистемы отображения.

Сущность parking_slot (парковочное место). На диаграмме показана отдельным блоком внизу: slot_id, label, points_norm. Отдельной таблицы в SQLite нет — координаты полигонов и метки slot_free / slot_occupied формируются конфигурацией приложения и результатом сегментации YOLOv8-seg. Связи с bookings, slot_overrides и observations отражают смысловую привязку по slot_id или содержимому slots_json, а не ссылочную целостность базы данных.

Связь между таблицами users и bookings является неидентифицирующей, так как бронь однозначно определяется первичным ключом bookings.id, и для идентификации экземпляров сущности bookings не нужен user_id. Связь один ко многим: один пользователь может иметь несколько броней, каждая бронь относится только к одному пользователю (bookings.user_id → users.id). Связь обязательная со стороны bookings так как у каждой брони обязательно должен быть пользователь; со стороны users необязательная — учётная запись может существовать без броней.

Связь между таблицами users и slot_overrides показана пунктирной линией с подписью JWT и не является внешним ключом SQLite — в slot_overrides нет столбца user_id. Связь отражает правило приложения, а не ссылочную целостность БД: изменение ручного статуса места доступно только администратору после проверки входа и роли admin. На уровне таблицы slot_overrides запись существует без привязки к пользователю; со стороны users участие в назначении override не фиксируется в базе данных.

Связь между таблицей bookings и логической сущностью parking_slot показана пунктирной линией logic и не является внешним ключом, поскольку отдельной таблицы слотов в базе нет. Связь многие к одному (со стороны броней): несколько записей могут относиться к одному и тому же slot_id (в разное время); каждая бронь привязана только к одному месту через строковое поле slot_id. Связь обязательная со стороны bookings, так как у каждой брони обязательно указано место slot_id; необязательная со стороны parking_slot — место может существовать в системе мониторинга без активной брони в таблице bookings. Идентификатор slot_id совпадает с id места из конфигурации и результата детекции.

Связь между таблицей slot_overrides и логической сущностью parking_slot также показана пунктиром logic и не является FK. Связь один к одному, не более одной записи override на место. Связь обязательная со стороны slot_overrides, так как каждая запись override обязательно относится к конкретному месту; необязательная со стороны parking_slot — для места override может не быть назначен. Override может быть установлен для любого slot_id, известного системе мониторинга.

Связь между таблицей observations и логической сущностью parking_slot показана пунктиром slots_json и не является FK. В поле slots_json одной строки observations хранится JSON-снимок множества мест на момент кадра, а не отдельная ссылка на одно место. Связь многие ко многим в смысле архива: одна запись observations описывает состояние сразу нескольких мест; одно и то же место встречается в разных строках. Со стороны observations связь с отдельным местом необязательна в виде FK — строка наблюдения не привязана к одному slot_id

### 3.2 Диаграмма вариантов использования

На рисунке 3.5 представлена диаграмма вариантов использования подсистемы «Отображение свободных и занятых мест» ИС «Умная парковка». Диаграмма показывает взаимодействие акторов с подсистемой в составе веб-приложения: просмотр карты мест и индикатора загрузки, бронирование и сопутствующие операции в личном кабинете, администрирование мест, пользователей, броней и аналитика. На диаграмме отражены все пятнадцать ключевых функций подсистемы (UC1–UC15). Доступ к операциям контролируется входом в систему и проверкой роли пользователя или администратора.

Акторы:

Пользователь (авторизованный) — выполняет операции в личном кабинете: просматривает карту мест и «светофор», оформляет и отменяет брони, рассчитывает стоимость, продлевает время, получает предупреждения, оплачивает парковку, просматривает историю и меняет пароль.

Администратор — работает в панели администратора: управляет учётными записями и бронями, назначает ручные статусы мест, анализирует загрузку парковки и финансовые показатели.

Основные варианты использования:

UC1. Отмена брони — пользователь в личном кабинете отменяет активную бронь одним действием; место снова может отображаться как доступное на карте, если нет других ограничений.

UC2. Тарифный калькулятор — при указании длительности стоянки система автоматически рассчитывает стоимость по почасовому тарифу и показывает ориентировочную сумму до подтверждения брони.

UC3. Продление времени парковки — пользователь увеличивает время окончания активной брони (например, на один час); при необходимости отображается сумма доплаты, список броней обновляется.

UC4. Уведомление об истечении времени — если до конца брони остаётся не более пятнадцати минут, в кабинете появляется предупреждение с номером места и оставшимся временем.

UC5. Личный кабинет, карта мест и «светофор» (базовый прецедент пользователя) — после входа пользователь видит приветствие, карту мест с объединённым состоянием (свободно, занято, забронировано, ручной статус) и индикатор загрузки парковки. Данные для карты формируются из распознавания с камеры, таблиц bookings и slot_overrides.

UC6. Смена пароля — пользователь вводит текущий и новый пароль; система проверяет правильность и сохраняет изменения без выхода из сеанса.

UC7. История поездок — в кабинете выводится список прошлых и текущих парковок: место, период, статус, госномер и сведения об оплате.

UC8. Оплата и чек — пользователь оплачивает неоплаченную бронь (демо-режим); формируется электронный чек с суммой и периодом стоянки, доступный для сохранения.

UC9. Панель администратора (базовый прецедент администратора) — после входа с ролью admin открывается сводка, списки пользователей и броней, карта мест, разделы аналитики и финансов.

UC10. Управление пользователями — администратор просматривает список учётных записей, создаёт новых пользователей, меняет имя и включает или отключает доступ.

UC11. Блокировка аккаунта с причиной — при отключении пользователя администратор указывает причину; заблокированный пользователь видит её при попытке входа.

UC12. Управление бронями (админ) — администратор просматривает все брони, переносит бронь на другое место или продлевает время стоянки.

UC13. Ручной статус места — администратор назначает месту статус «ремонт», «резерв» или «закрыто»; место не предлагается для брони и отображается с приоритетной пометкой независимо от камеры.

UC14. Аналитика загрузки — по накопленным данным мониторинга формируется отчёт о занятости парковки по часам и по дням за выбранный период.

UC15. Финансовая аналитика — за период отображаются суммарная выручка, число оплат, средний чек и действующий тариф.

Вспомогательный прецедент UC0. Авторизация — вход в систему, создание сеанса и проверка прав доступа; обязателен для UC1–UC15.

Связи include и extend:

Include: все прецеденты UC1–UC15 включают UC0 (авторизация). Для UC9–UC15 дополнительно требуется роль администратора.

Extend: UC1–UC8 расширяют UC5 как опциональные сценарии в личном кабинете после просмотра карты мест и списка броней. UC10–UC15 расширяют UC9 как опциональные разделы панели администратора.

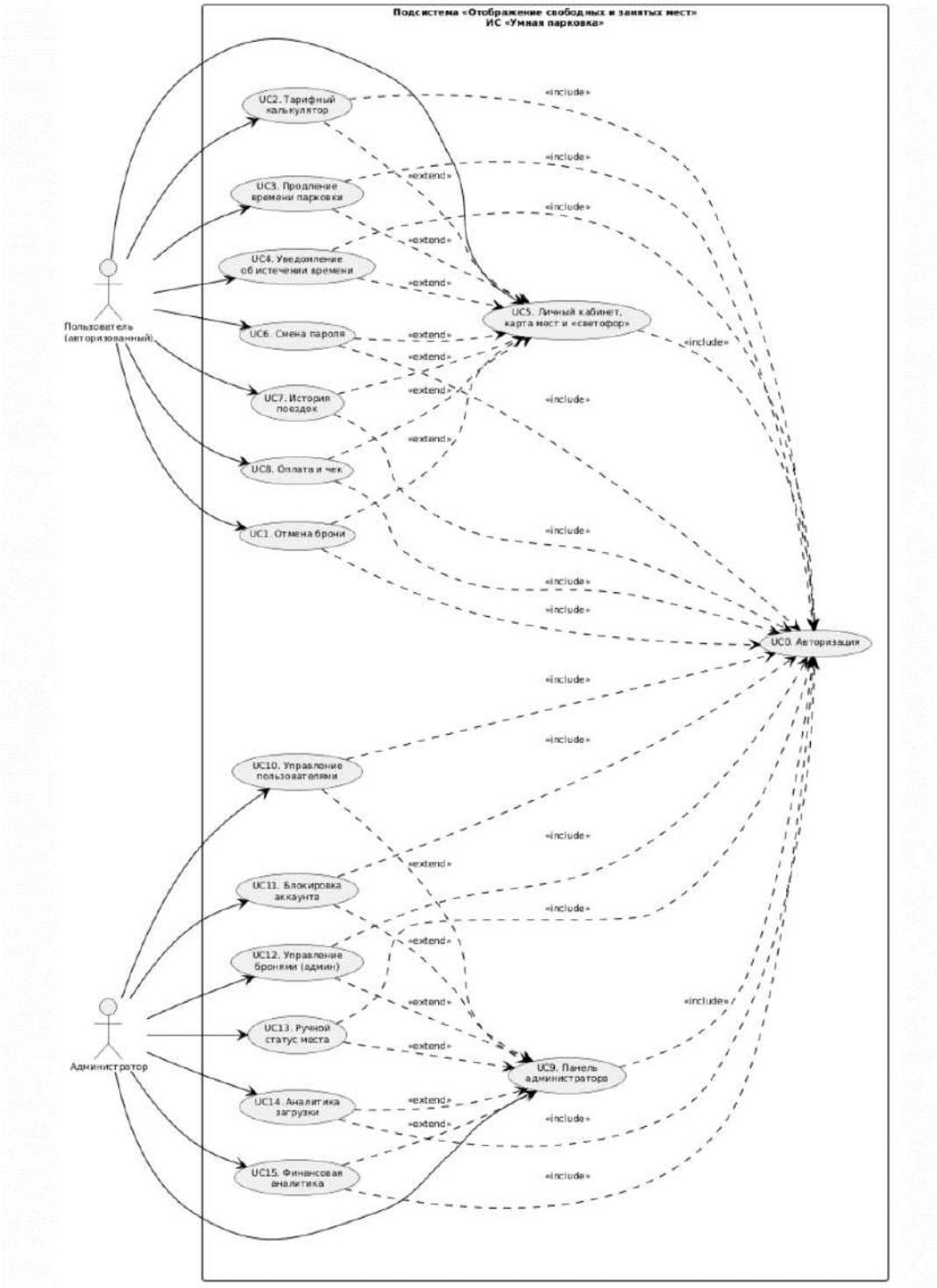



Рисунок 3.5 – Диаграмма вариантов использования подсистемы «Отображение свободных и занятых мест».

### 3.3 Диаграммы последовательностей

На рисунках 3.6–3.9 представлены четыре диаграммы последовательностей, объединяющие ключевые сценарии подсистемы. Диаграммы отражают обмен сообщениями между интерфейсом (JavaScript в браузере), сервером (FastAPI, server.py), SQLite (parking.db) и оперативным снимком детекции (_last_slots). Ниже для каждого рисунка указаны покрываемые функции и пошаговое описание. На рисунке 3.6 сценарий видеодетекции (основа для функции 5 и мониторинга). Покрываемые функции: подготовка данных для карты мест и «светофора» (функция 5, часть «фактическая занятость по кадру»). Пользователь на странице «Мониторинг» захватывает кадр (камера или файл) и инициирует отправку. UI (app.js) выполняет POST /api/detect с изображением. Сервер запускает YOLOv8-seg, определяет метки slot_free / slot_occupied, присваивает стабильные id слотов и применяет сглаживание (debounce). Сервер обновляет live-состояние (_last_slots, _last_ts) в памяти. При необходимости выполняется фоновая запись в таблицу observations. Сервер возвращает JSON (slots, counts); UI перерисовывает overlay и счётчики.

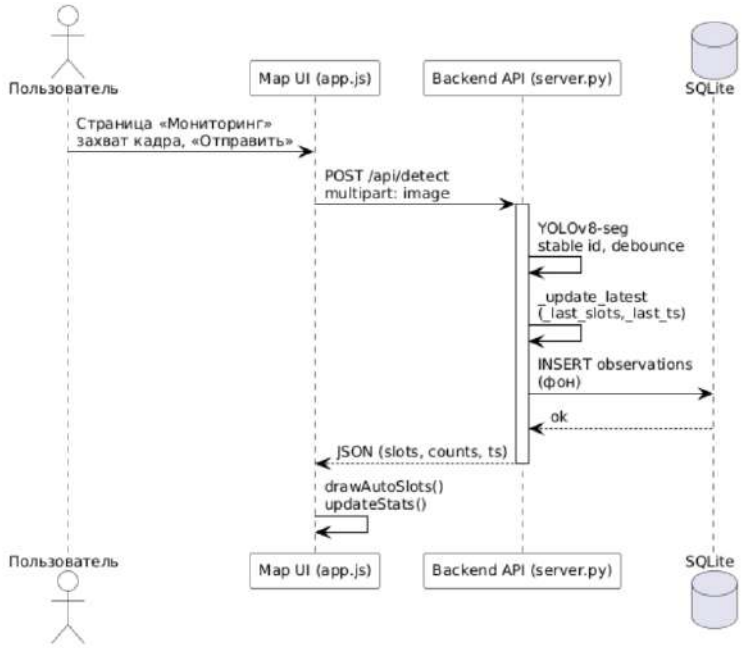



Рисунок 3.6 – Диаграмма последовательности «Детекция слотов на кадре (POST /api/detect)».

На рисунке 3.7 — просмотр объединённой карты мест в личном кабинете. Покрываемые функции: 5 (карта мест и «светофор»), 4 (данные для предупреждения об истечении — remaining_s в том же сеансе кабинета). Шаг 1. Авторизованный пользователь открывает cabinet.html. UI запрашивает GET /api/parking/slots-map (cookie

pe_session). Сервер проверяет сеанс (require_login). Сервер читает активные брони из bookings и ручные статусы из slot_overrides. Сервер объединяет данные с _last_slots: для каждого слота вычисляются detected, state, bookable и блок traffic. UI отрисовывает сетку карты (renderSlotMap) и индикатор загрузки; при remaining_s ≤ 15 мин может показаться баннер (функция 4).

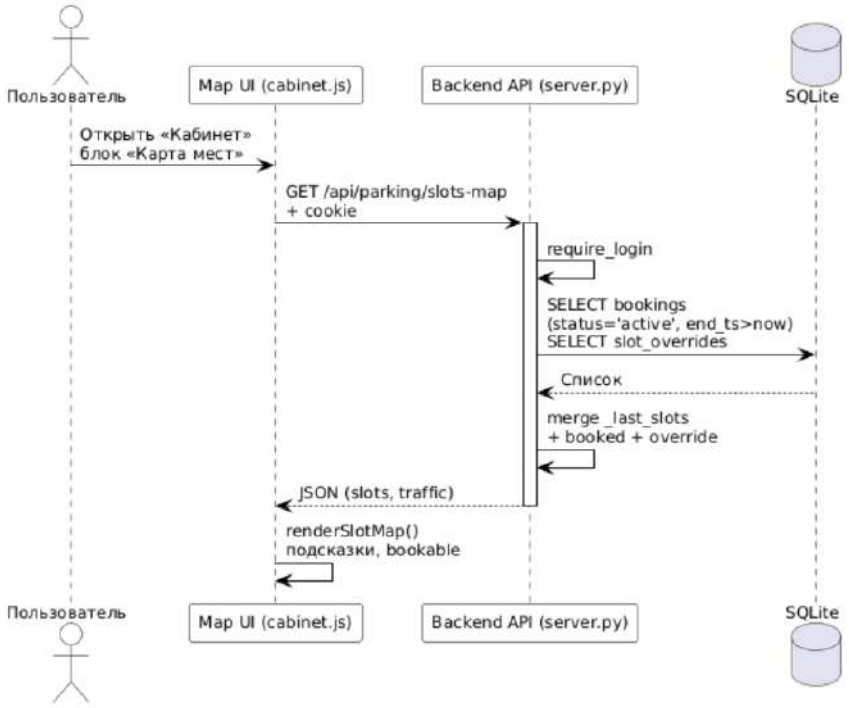



Рисунок 3.7 – Диаграмма последовательности «Просмотр карты мест (GET /api/parking/slots-map)».

На рисунке 3.8 изображена диаграмма последовательности «Бронирование и управление бронями в кабинете». Покрываемые функции: тарифный калькулятор, создание брони, отмена, продление, оплата и чек. Пользователь выбирает свободное место на карте; в форме указывает часы и госномер. При изменении длительности UI вызывает POST /api/tariff/calc — отображается расчётная сумма. По кнопке «Забронировать» UI отправляет POST /api/me/bookings. Сервер проверяет bookable(slot_id) с учётом override, брони и детекции; выполняет INSERT в bookings. UI обновляет список «Мои брони» и карту (место — booked). Далее в том же интерфейсе: DELETE — отмена, PATCH с extend_hours — продление, POST .../pay — оплата и чек.

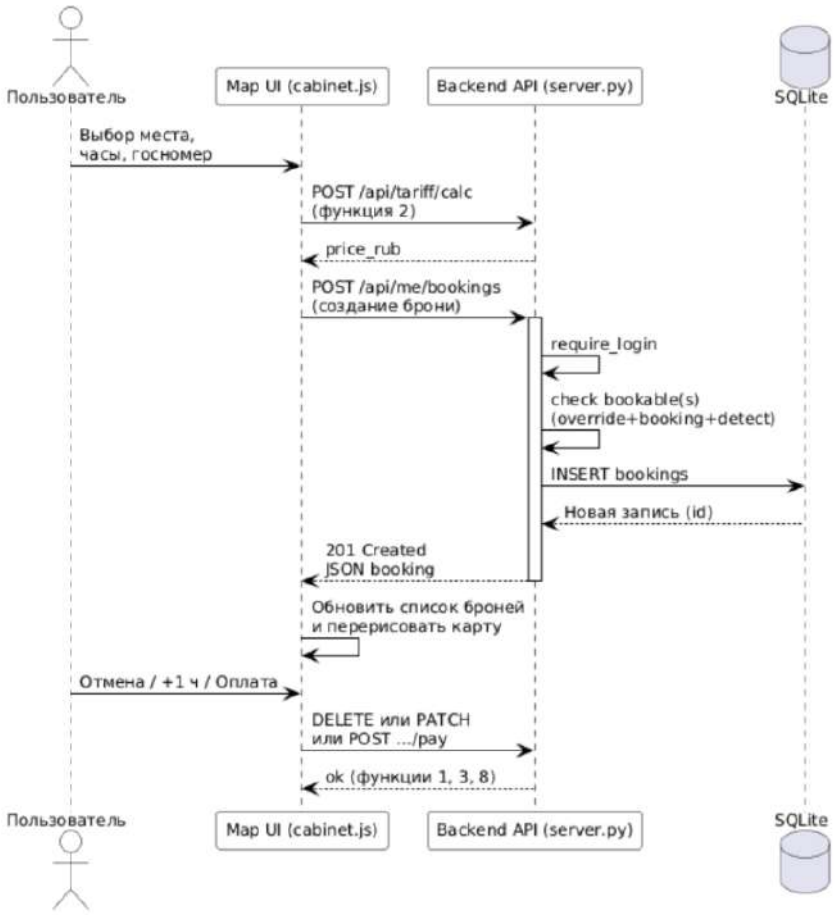



Рисунок 3.8 – Диаграмма последовательности «Бронирование и управление бронями в кабинете».

На рисунке 3.9 изображена диаграмма последовательности «Администрирование: статус места и аналитика». Покрываемые функции: панель администратора, ручной статус места, аналитика загрузки, финансовая аналитика, пользователи, блокировка, все брони. Администратор входит с ролью admin и открывает admin.html. UI загружает сводку GET /api/admin/summary. На админ-карте клик по ячейке — PUT /api/admin/slot-overrides/{slot_id} со статусом closed или repair; запись UPSERT в slot_overrides. UI запрашивает GET /api/parking/slots-map — место с bookable: false. В разделах «Аналитика» и «Финансы» UI запрашивает GET /api/admin/analytics/occupancy и GET /api/admin/finance/summary. Сервер на каждом шаге проверяет require_admin; данные отображаются в таблицах и сводках админ-панели.

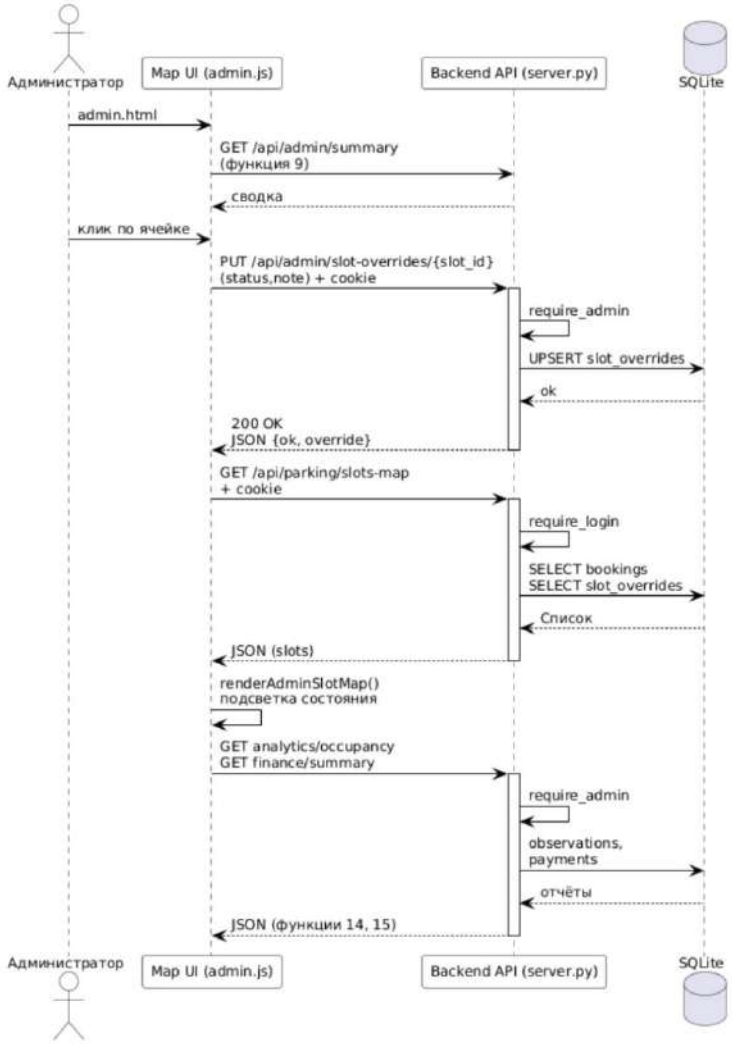



Рисунок 3.9 – Диаграмма последовательности «Администрирование: статус места и аналитика».

## 4. Разработка документации приложения

### 4.1 Выбор и обоснование технологий, используемых в программном модуле

Для реализации подсистемы «Отображение свободных и занятых мест» в составе ИС «Умная парковка» выбрана клиент-серверная архитектура с веб-интерфейсом, REST API и реляционной базой данных. Выбор технологий выполнен с учётом требований индивидуальной темы: мониторинг парковки по видеокадру с наложением результатов детекции, отображение карты мест в личном кабинете, вычисление индикатора загрузки («светофор»), объединение источников состояния (детекция slot_free/slot_occupied, активные брони bookings и ручные статусы slot_overrides), а также административные операции (отключение места и поиск по госномеру). При выборе стека также учитывалась необходимость быстрого развёртывания учебного проекта и удобство сопровождения.

Python используется как основной язык серверной части подсистемы. На нём реализованы обработчики HTTP-запросов, проверка прав доступа, бизнес-правила объединения state(s) и bookable(s), а также интеграция с моделью детекции. Выбор Python обоснован наличием зрелой экосистемы для веб-API и компьютерного зрения и высокой скоростью разработки в учебном проекте.

FastAPI применяется как веб-фреймворк для построения REST API. Через FastAPI объявлены конечные точки POST /api/detect (детекция), GET /api/parking/slots-map (карта мест), GET /api/parking/traffic (индикатор загрузки), POST/GET /api/me/bookings (бронирования) и административные операции PUT /api/admin/slot-overrides/{slot_id}, GET /api/admin/find-by-plate. Кроме того, платформа предоставляет эндпоинты регистрации и входа (/api/auth/*), что обеспечивает единый контур аутентификации для всех страниц. Выбор FastAPI обусловлен высокой производительностью, встроенной валидацией входных данных и удобной организацией зависимостей (Depends) для авторизации и разграничения ролей.

SQLite выбрана в качестве СУБД проекта (файл parking.db). В подсистеме используются таблицы bookings (активные брони), slot_overrides (ручные статусы мест), users и sessions (аутентификация по cookie), а также observations (архив агрегатов детекции). SQLite обеспечивает целостность данных через первичные ключи и внешние ключи (например, bookings.user_id → users.id), выполнение выборок активных броней и обновление ручных статусов мест (UPSERT slot_overrides). Выбор SQLite обоснован простотой развёртывания в учебном проекте, поддержкой транзакций и достаточной производительностью для требуемых операций чтения/записи.

JavaScript используется на клиентской стороне для реализации поведения страниц мониторинга, кабинета и админ-панели. Модули app.js, cabinet.js, admin.js, pe-features.js выполняют отрисовку overlay и карты мест, вызывают API через единый адаптер Auth.fetchJson (cookie-сессии) и обновляют интерфейс по JSON-ответам. Выбор JavaScript обоснован нативной поддержкой в браузере и прямой интеграцией с HTML/CSS.

HTML/CSS используются для разметки страниц (index.html, cabinet.html, admin.html) и единого оформления панелей и карты мест (styles.css). Такой подход позволяет отделить представление от логики и упростить сопровождение.

JSON используется как базовый формат обмена данными между клиентом и сервером. Через JSON передаются элементы слотов (id, detected, state, override, bookable), данные бронирования и результаты административных операций.

Cookie-сессии (pe_session) применяются для авторизации пользователей без передачи токена в каждом запросе. Сервер хранит хэш токена сессии в таблице sessions, а браузер автоматически отправляет cookie при вызовах API через fetch с credentials. Такой подход упрощает интеграцию страниц кабинета и админ-панели и позволяет разграничить роли user/admin через require_login и require_admin.

YOLOv8-seg (модель сегментации Ultralytics) используется для детекции парковочных мест на кадре. Результат детекции представляется в виде полигонов в нормализованных координатах (0..1) и меток slot_free/slot_occupied; далее сервер присваивает стабильные идентификаторы местам и применяет сглаживание меток (debounce). Выбор модели обоснован практической применимостью для задач компьютерного зрения и качеством сегментации.

Таким образом, выбранный технологический стек обеспечивает реализацию всех ключевых функций подсистемы: получение детекции слотов, расчёт индикатора загрузки, объединение статусов и отображение карты мест, создание броней и административное управление доступностью мест.

### 4.2 Описание используемых структур данных

На рисунке 4.1 представлена диаграмма компонентов пользовательского интерфейса подсистемы. Пользователь взаимодействует со страницами index.html (мониторинг), cabinet.html (кабинет) и admin.html (администрирование). Разметка страниц отделена от программной логики: поведение интерфейса реализовано в JavaScript-модулях app.js (video + canvas overlay, отправка кадра на детекцию), cabinet.js (загрузка карты мест и бронирование), admin.js (управление slot_overrides и поиск по госномеру) и pe-features.js (отрисовка карты мест и индикатора traffic).

Модуль auth.js выделен отдельным компонентом: он выполняет запросы к API с учётом cookie-сессии (pe_session), предоставляет функцию Auth.fetchJson и

обрабатывает сценарии входа/выхода. Компонент app-shell.js обеспечивает единый каркас интерфейса (навигация, доступ к админ-разделу по роли). Компоненты settings.js и styles.css обеспечивают настройки (i18n, темы) и единообразное оформление.

Все обращения UI к backend сведены к группам эндпоинтов подсистемы: /api/detect, /api/parking/slots-map, /api/parking/traffic, /api/me/bookings, /api/admin/slot-overrides и /api/admin/find-by-plate. Это позволяет изменять формат хранения в SQLite или правила объединения state(s), не перерабатывая разметку страниц, если сохраняются контракты JSON.

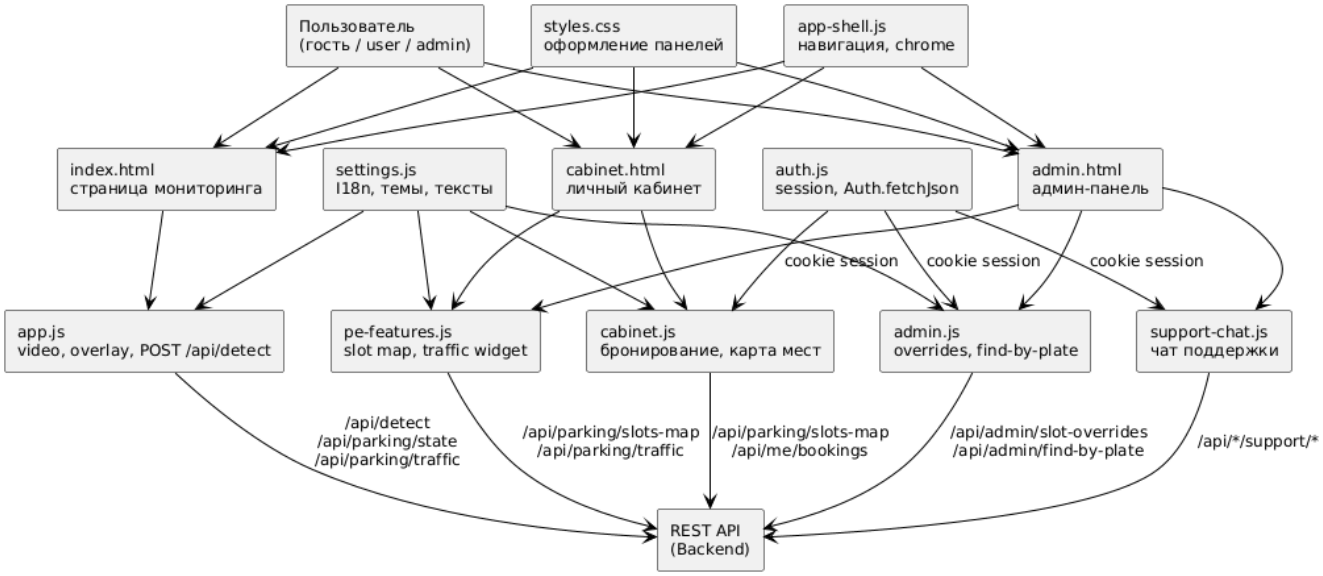



Рисунок 4.1 – Диаграмма компонентов пользовательского интерфейса подсистемы «Отображение свободных и занятых мест».

На рисунке 4.2 представлена диаграмма компонентов серверной части подсистемы. Сервер реализован как единое приложение FastAPI (server.py). Для подсистемы выделены логические группы обработчиков: детекция (POST /api/detect), парковочные сводки и карта (GET /api/parking/traffic, GET /api/parking/slots-map), бронирования (/api/me/bookings) и административные операции (/api/admin/slot-overrides, /api/admin/find-by-plate).

Проверка прав реализована через зависимости require_login и require_admin (cookie-сессии). Все обработчики используют единый слой доступа к данным (_db(), _db_lock) и SQLite. Важно, что live-детекция хранится в памяти процесса (_last_slots) и объединяется с данными БД в _parking_slots_map_payload, а таблица observations служит для истории и аналитики.

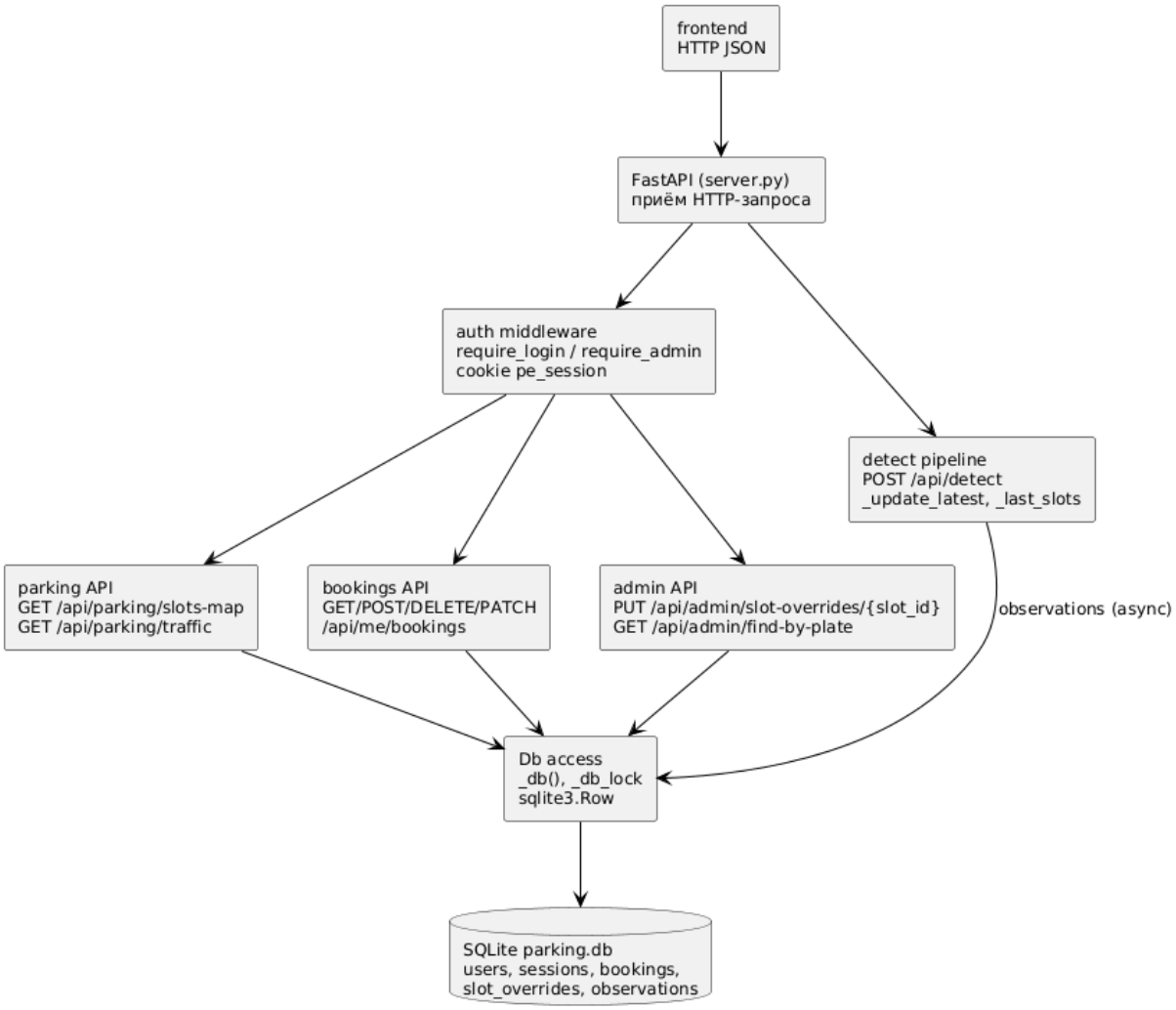



Рисунок 4.2 – Диаграмма компонентов серверной части подсистемы «Отображение свободных и занятых мест».

На таблицах 4.1–4.3 приведено описание связи компонентов, назначение компонентов и передача данных по эндпоинтам. Таблицы позволяют проследить полный маршрут прохождения управления в подсистеме «Отображение свободных и занятых мест»: от действий пользователя на страницах мониторинга/кабинета/админки до вызовов REST API и операций чтения/записи в SQLite.

Таблица 4.1 – Предоставлено описание связи компонентов.

| От | К | Описание |
|---|---|---|
| Пользователь | index.html | Открытие мониторинга (видео + overlay) и просмотр счётчиков. |
| Пользователь | cabinet.html | Открытие кабинета: карта мест, форма брони, список броней. |
| Администратор | admin.html | Открытие админ-панели: сводка, поиск по госномеру, статусы мест. |

Продолжение таблицы 4.1

| От | К | Описание |
|---|---|---|
| index.html | app.js | Логика мониторинга: камера/поток, отрисовка overlay, вызов POST /api/detect. |
| cabinet.html | cabinet.js | Логика кабинета: загрузка slots-map, создание/отмена брони. |
| admin.html | admin.js | Логика админки: PUT slot-overrides, GET find-by-plate, обновление карты. |
| pe-features.js | server.py | GET /api/parking/slots-map и /api/parking/traffic для карты и «светофора». |
| auth.js | server.py | Сессия и роли: /api/auth/*, cookie pe_session, доступ require_login/admin. |
| server.py | SQLite parking.db | Запросы к bookings, slot_overrides, users, sessions, observations. |
| app.js | server.py | POST /api/detect; обновление _last_slots; запись observations (фон). |

Таблица 4.1 показывает маршруты взаимодействия: пользователь работает со страницами интерфейса, клиентские модули JavaScript формируют HTTP-запросы и получают JSON-ответы, а сервер выполняет проверку доступа и обращается к таблицам SQLite. Такой подход исключает обращение интерфейса к базе данных напрямую и обеспечивает единые контракты API для всех сценариев.

Таблица 4.2 – представлено описание компонентов.

| Уровень | Компонент | Назначение |
|---|---|---|
| Frontend | index.html | Страница мониторинга: video, canvas overlay, счётчики, режимы source/client. |
| Frontend | cabinet.html | Кабинет пользователя: карта мест, форма брони, список броней. |
| Frontend | admin.html | Админ-панель: поиск по госномеру, ручные статусы мест, сводка. |
| Frontend | static/app.js | Захват кадра и POST /api/detect, отрисовка overlay. |
| Frontend | static/pe-features.js | Виджеты traffic и карты мест (renderTraffic, renderSlotMap). |
| Frontend | static/cabinet.js | Сценарии бронирования и обновления карты мест. |
| Frontend | static/admin.js | Сценарии администрирования slot_overrides и поиска по plate. |
| Frontend | static/auth.js | Auth.fetchJson, login/register/logout, cookie-сессия. |
| Backend | server.py — /api/detect | Детекция слотов, stable id, debounce, обновление live-состояния. |
| Backend | server.py — /api/me/bookings | Создание/список/отмена/изменение броней. |

Продолжение таблицы 4.2

| Уровень | Компонент | Назначение |
|---|---|---|
| Backend | server.py — /api/parking/slots-map, /traffic | Выдача карты мест и индикатора загрузки. |
| Backend | server.py — /api/admin/* | Админ-операции: slot_overrides, find-by-plate, сводка. |
| Data Storage | SQLite parking.db | users, sessions, bookings, slot_overrides, observations. |

Таблица 4.2 фиксирует распределение компонентов по уровням. На стороне клиента выделены модули, обеспечивающие индивидуальные сценарии: overlay детекции, карта мест и администрирование статусов. На стороне сервера логические группы обработчиков соответствуют эндпоинтам подсистемы и используют единый слой доступа к данным и live-состояние детекции (_last_slots).

Таблица 4.3 – представлено описание передачи данных по эндпоинтам

| Компонент | Метод | Эндпоинт | Тело запроса | Ответ |
|---|---|---|---|---|
| app.js | POST | /api/detect | multipart: image | JSON: slots(points,label,conf), counts, ts |
| pe-features.js/cabinet.js | GET | /api/parking/slots-map | — | JSON: slots(id,detected,state,override,bookable), traffic |
| pe-features.js | GET | /api/parking/traffic | — | JSON: level, pct_free, free,total,occupied |
| cabinet.js | GET | /api/me/bookings | — | JSON: список броней + remaining_s |
| cabinet.js | DELETE | /api/me/bookings/{id} | — | 200/ok (status cancelled) |
| admin.js | PUT | /api/admin/slot-overrides/{slot_id} | JSON: status, note? | 200/ok + override |
| admin.js | GET | /api/admin/find-by-plate?plate= | — | JSON: slot_id, booking + live_state |

Таблица 4.3 отражает контракт обмена данными: детекция передаёт полигоны слотов и метки, карта мест получает объединённый state(s) и traffic, а операции бронирования и административные изменения приводят к записи в таблицы.

## 5 Тестирование приложения

Тестирование подсистемы «Отображение свободных и занятых мест» выполнено с использованием фреймворка pytest и тестового клиента веб-приложения. Составлено пятнадцать автоматических тестов — по одному на каждую ключевую функцию подсистемы (раздел 2.2). Проверяются операции личного кабинета и админ-панели, работа с базой данных SQLite и контроль доступа по ролям.

Тест 1 — «Отмена брони в один клик». Создаётся активная бронь, затем выполняется её отмена. Проверяется, что статус записи в списке броней меняется на cancelled. Результат: PASS.

Тест 2 — «Тарифный калькулятор». Отправляется запрос расчёта стоимости на два часа; проверяется, что сумма равна удвоенному почасовому тарифу. Результат: PASS.

Тест 3 — «Продление времени парковки». Для активной брони выполняется продление на один час; проверяется увеличение времени окончания. Результат: PASS.

Тест 4 — «Уведомление об истечении времени». Создаётся бронь на 15 минут; проверяется, что remaining_s ≤ 900 сек — данных достаточно для баннера в кабинете. Результат: PASS.

Тест 5 — «Личный кабинет, карта мест и светофор». Проверяются профиль, сводка кабинета, карта мест и индикатор загрузки. Результат: PASS.

Тест 6 — «Смена пароля». Пароль меняется, старый не принимается, новый принимается; пароль восстанавливается. Результат: PASS.

Тест 7 — «История поездок». Проверяется список sessions с записями о бронированиях. Результат: PASS.

Тест 8 — «Оплата и чек». Демо-оплата брони; проверяется номер чека receipt_id. Результат: PASS.

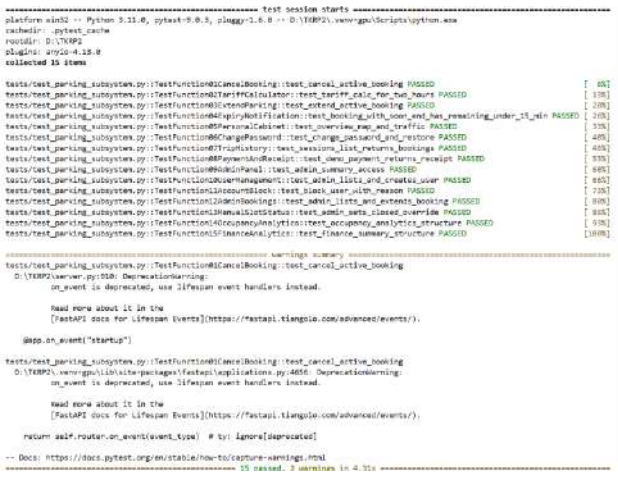



Рисунок 5.1 – Автоматическое тестирование функций 1–8 (личный кабинет)

Тест 9 — «Панель администратора». Пользователю — отказ; администратору — сводка. Результат: PASS.

Тест 10 — «Управление пользователями». Список учётных записей и создание нового пользователя. Результат: PASS.

Тест 11 — «Блокировка аккаунта с причиной». Отключение с причиной; вход заблокирован. Результат: PASS.

Тест 12 — «Управление бронями (админ)». Бронь в общем списке и продление на 30 мин. Результат: PASS.

Тест 13 — «Ручной статус места». Статус closed в slot_overrides. Результат: PASS.

Тест 14 — «Аналитика загрузки». Отчёт за 7 дней (hourly, daily). Результат: PASS.

Тест 15 — «Финансовая аналитика». Сводка revenue_rub и tariff_hour_rub. Результат: PASS.

```
============================= test session starts ==============================
platform win32 -- Python 3.11.0, pytest-9.0.3, pluggy-1.6.0 -- D:\TKRP2\.venv-gpu\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\TKRP2
plugins: anyio-4.13.0
collected 15 items

tests/test_parking_subsystem.py::TestFunction01CancelBooking::test_cancel_active_booking PASSED [  6%]
tests/test_parking_subsystem.py::TestFunction02TariffCalculator::test_tariff_calc_for_two_hours PASSED [ 13%]
tests/test_parking_subsystem.py::TestFunction03ExtendParking::test_extend_active_booking PASSED [ 20%]
tests/test_parking_subsystem.py::TestFunction04ExpiryNotification::test_booking_with_soon_end_has_remaining_under_15_min PASSED [ 26%]
tests/test_parking_subsystem.py::TestFunction05PersonalCabinet::test_overview_map_and_traffic PASSED [ 33%]
tests/test_parking_subsystem.py::TestFunction06ChangePassword::test_change_password_and_restore PASSED [ 40%]
tests/test_parking_subsystem.py::TestFunction07TripHistory::test_sessions_list_returns_bookings PASSED [ 46%]
tests/test_parking_subsystem.py::TestFunction08PaymentAndReceipt::test_demo_payment_returns_receipt PASSED [ 53%]
tests/test_parking_subsystem.py::TestFunction09AdminPanel::test_admin_summary_access PASSED [ 60%]
tests/test_parking_subsystem.py::TestFunction10UserManagement::test_admin_lists_and_creates_user PASSED [ 66%]
tests/test_parking_subsystem.py::TestFunction11AccountBlock::test_block_user_with_reason PASSED [ 73%]
tests/test_parking_subsystem.py::TestFunction12AdminBookings::test_admin_lists_and_extends_booking PASSED [ 80%]
tests/test_parking_subsystem.py::TestFunction13ManualSlotStatus::test_admin_sets_closed_override PASSED [ 86%]
tests/test_parking_subsystem.py::TestFunction14OccupancyAnalytics::test_occupancy_analytics_structure PASSED [ 93%]
tests/test_parking_subsystem.py::TestFunction15FinanceAnalytics::test_finance_summary_structure PASSED [100%]

============================= warnings summary =============================
tests/test_parking_subsystem.py::TestFunction01CancelBooking::test_cancel_active_booking
  D:\TKRP2\server.py:910: DeprecationWarning:
        on_event is deprecated, use lifespan event handlers instead.

        Read more about it in the
        [FastAPI docs for Lifespan Events](https://fastapi.tiangolo.com/advanced/events/).

    @app.on_event("startup")

tests/test_parking_subsystem.py::TestFunction01CancelBooking::test_cancel_active_booking
  D:\TKRP2\.venv-gpu\Lib\site-packages\fastapi\applications.py:4656: DeprecationWarning:
        on_event is deprecated, use lifespan event handlers instead.

        Read more about it in the
        [FastAPI docs for Lifespan Events](https://fastapi.tiangolo.com/advanced/events/).

    return self.router.on_event(event_type)  # ty: ignore[deprecated]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================== 15 passed, 2 warnings in 4.31s ========================
```

Рисунок 5.2 – Автоматическое тестирование функций 9–15 (админ-панель)

Бронирование и уведомления. Тесты 1–4 проверяют отмену активной брони, расчёт стоимости по тарифу, продление времени парковки и подготовку данных для предупреждения об истечении срока (остаток времени не более пятнадцати минут). Все четыре теста завершились с результатом PASS.

Личный кабинет. Тест 5 проверяет вход в кабинет, отображение профиля, карту мест и индикатор «светофор» (уровень загрузки green / yellow / red). Тесты 6–8 проверяют смену пароля, вывод истории поездок и демо-оплату с формированием номера чека. Результат всех четырёх тестов — PASS.

Администрирование. Тесты 9–13 проверяют доступ к админ-панели (отказ обычному пользователю и доступ администратору), создание учётных записей, блокировку с указанием причины, просмотр и продление броней администратором, а также запись ручного статуса места в таблицу slot_overrides. Результат — PASS.

Аналитика. Тесты 14–15 проверяют отчёт занятости парковки по часам и дням и финансовую сводку (выручка, число оплат, тариф). Результат обоих тестов — PASS.

Итого выполнено 15 автоматических тестов — по одному на каждую ключевую функцию подсистемы; все завершились с результатом PASS.

Дополнительно выполнено ручное тестирование по четырём сценариям, отражающим основные пользовательские и административные операции подсистемы. Для каждого сценария зафиксированы предусловие, действия испытателя, ожидаемый и фактический результат; иллюстрации — на рисунках 5.3–5.6.

Сценарий Р-1. Личный кабинет и карта мест.

Предусловие: в базе данных есть записи о бронированиях и/или результаты видеомониторинга; испытатель входит под учётной записью пользователя (например, ivan) и открывает страницу личного кабинета (cabinet.html). Система загружает карту мест (GET /api/parking/slots-map) и отображает приветствие по имени, навигацию по разделам и сетку парковочных ячеек с цветовой индикацией состояний (свободно, занято, забронировано). Ожидается корректное отображение профиля и актуальных статусов мест на карте (функция 5). Фактический результат: соответствует ожидаемому. На рисунке 5.3 зафиксированы приветствие и карта мест

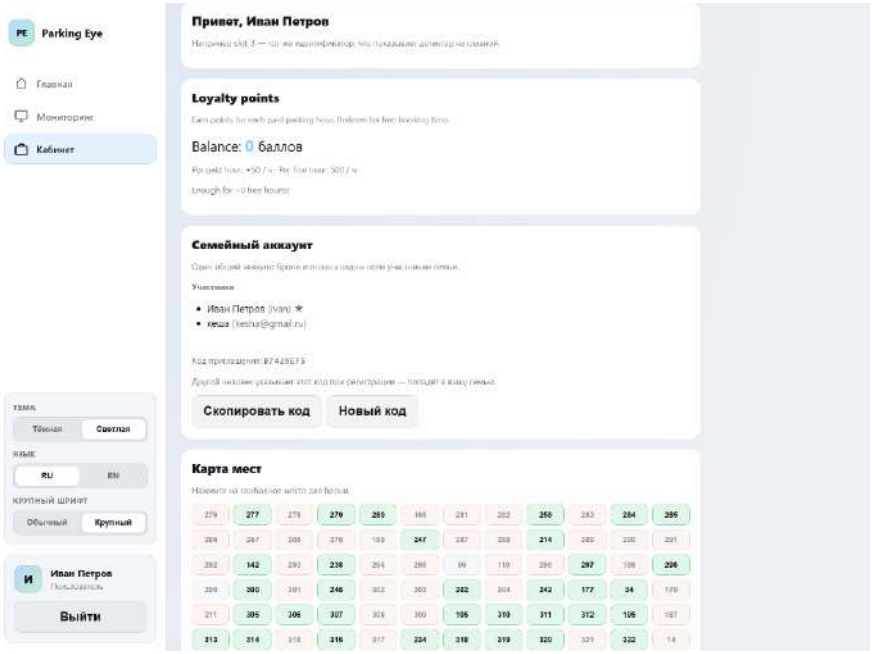



Рисунок 5.3 –Ручное тестирование Сценария Р-1

Сценарий Р-2. Бронирование: форма, список и управление бронями.

Предусловие: пользователь авторизован, на карте есть свободные места (bookable: true). Испытатель в блоке «Бронирование» указывает длительность стоянки — система рассчитывает стоимость по тарифу (POST /api/tariff/calc) и показывает сумму рядом с формой (функция 2). После подтверждения создаётся бронь (POST /api/me/bookings); в списке активных броней отображаются место, период и команды «Отменить» и «+1 ч» (функции 1 и 3). При необходимости выполняется демо-оплата неоплаченной брони с формированием чека (POST /api/me/bookings/{id}/pay, функция 8). Ожидается обновление списка броней и согласованность данных с картой мест. Фактический результат: соответствует ожидаемому. Представлено на рисунке 5.4.

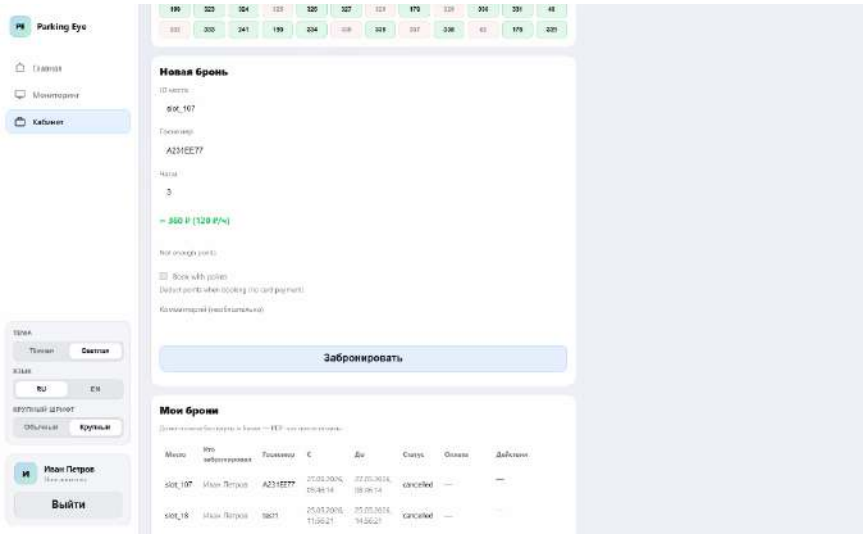



Рисунок 5.4 – Ручное тестирование Сценарий Р-2

Сценарий Р-3. Администрирование: ручной статус места.

Предусловие: в базе данных есть активные брони или записи наблюдений. Испытатель входит под ролью admin и открывает админ-панель (admin.html). На админ-карте для выбранного slot_id устанавливается ручной статус closed или repair (PUT /api/admin/slot-overrides/{slot_id}) — место становится недоступным для бронирования (bookable: false в ответе GET /api/parking/slots-map) и отображается с соответствующей пометкой «закрыто» или «ремонт» (функция 13). Ожидается, что ячейка на админ-карте и на пользовательской карте отражает назначенный статус. Фактический результат: соответствует ожидаемому. Представлено на рисунке 5.5.

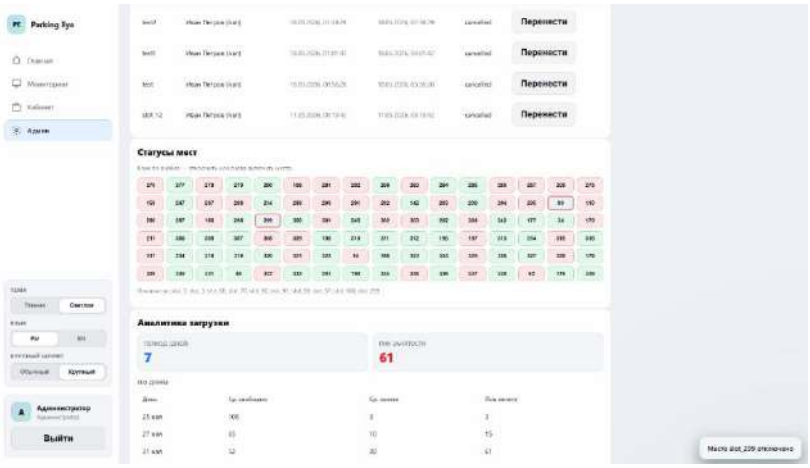


Рисунок 5.5 – Сценарий Р– 3

Сценарий Р-4. Администрирование: аналитика загрузки и финансы.

Предусловие: в системе накоплены данные о бронированиях, оплатах и мониторинге. Администратор в админ-панели открывает раздел «Аналитика» — за выбранный период отображаются сводка занятости парковки по дням и часам (GET /api/admin/analytics/occupancy, функция 14). Затем переходит в раздел «Финансы» — выводятся суммарная выручка, число оплат, средний чек и действующий почасовой тариф (GET /api/admin/finance/summary, функция 15). Фактический результат: соответствует ожидаемому. Представлено на рисунке 5.6.

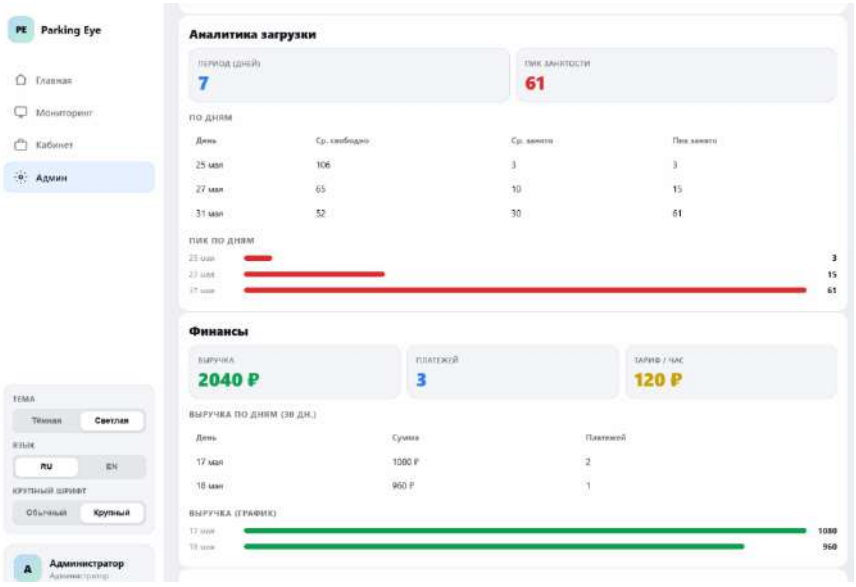



Рисунок 5.6 – Сценарий Р – 4

Остальные функции подсистемы проверены в ходе работы с кабинетом и админ-панели либо автоматическими тестами. Расхождений между ожидаемым и фактическим поведением не выявлено.

## Заключение

Тема курсовой работы: разработка информационной системы мониторинга и управления парковочными местами — ИС «Умная парковка».

Индивидуальное задание: в составе командной ИС разработать подсистему «Отображение свободных и занятых мест», включающую проектирование модели данных и REST API, отображение статусов мест на интерактивной карте в личном кабинете, получение актуального состояния по результатам видеодетекции (YOLOv8-seg), формирование индикатора загрузки парковки («светофор»), учёт активных бронирований и ручных статусов мест, а также административные функции отключения места и поиска автомобиля по государственному регистрационному номеру.

В ходе выполнения курсовой работы в рамках указанной темы и индивидуального задания реализована подсистема «Отображение свободных и занятых мест» в составе ИС «Умная парковка». Спроектированы модель данных и REST API; обеспечено отображение карты мест с признаками detected, state, bookable и override; реализован расчёт уровня загрузки; добавлены бронирования пользователем и административное управление доступностью мест с поиском по госномеру.

Подсистема реализована в едином контуре «веб-клиент (HTML/CSS/JavaScript) — серверная часть на Python (FastAPI) — SQLite», что обеспечивает целостность архитектуры и удобство дальнейшего развития. Для изменяющих операций с бронями, тарифом и административными данными предусмотрено разграничение доступа по ролям; результаты детекции и объединённое состояние мест отдаются клиенту через эндпоинты.

Практическая ценность работы заключается в объединении в одном интерфейсе сведений о фактической занятости мест (по видеокадру), логических статусах (бронь, ручное закрытие) и агрегированном индикаторе загрузки, что упрощает выбор свободного места пользователем и контроль парковки администратором. По сравнению с аналогами проект выделяется сочетанием компьютерного зрения на стороне сервера, ролевой модели, возможностью масштабирования API и ориентацией на задачи учебного прототипа умной парковки.

Результаты реализации и тестирования (автоматизированные испытания pytest и FastAPI TestClient — 15 тестов по ключевым функциям подсистемы; четыре ручных сценария Р-1–Р-4 по плану испытаний, иллюстрированные на рисунках 5.3–5.6) подтверждают работоспособность принятых решений и соответствие поставленным требованиям индивидуальной части. Перспективы развития включают подключение нескольких камер, оптимизацию частоты обновления карты мест и усиление безопасности при переходе к промышленной эксплуатации.

## Список литературы


1. SQLite: Documentation [Электронный ресурс]. Режим доступа: https://www.sqlite.org/docs.html (дата обращения: 21.03.2026).
2. Python: официальная документация [Электронный ресурс]. Режим доступа: https://docs.python.org/3/ (дата обращения: 27.03.2026).
3. FastAPI: Documentation [Электронный ресурс]. Режим доступа: https://fastapi.tiangolo.com/ (дата обращения: 13.03.2026).
4. Uvicorn — ASGI-сервер для Python [Электронный ресурс]. Режим доступа: https://www.uvicorn.org/ (дата обращения: 11.03.2026).
5. Ultralytics YOLO: Documentation [Электронный ресурс]. Режим доступа: https://docs.ultralytics.com/ (дата обращения: 07.04.2026).
6. OpenCV: Documentation [Электронный ресурс]. Режим доступа: https://docs.opencv.org/ (дата обращения: 28.02.2026).
7. MDN Web Docs: JavaScript [Электронный ресурс]. Режим доступа: https://developer.mozilla.org/ru/docs/Web/JavaScript (дата обращения: 01.04.2026).
8. JSON: спецификация (ECMA-404) [Электронный ресурс]. Режим доступа: https://www.ecma-international.org/publications-and-standards/standards/ecma-404/ (дата обращения: 29.02.2026).
9. pytest: Documentation [Электронный ресурс]. Режим доступа: https://docs.pytest.org/ (дата обращения: 05.03.2026).
10. JSON: спецификация (ECMA-404) [Электронный ресурс]. Режим доступа: https://www.ecma-international.org/publications-and-standards/standards/ecma-404/ (дата обращения: 05.03.2026).
11. Гринберг, М. Разработка веб-приложений на Flask / М. Гринберг; пер. с англ. — 2-е изд. — Санкт-Петербург: Питер, 2020. — 336 с.
12. Рашка, С. Python и машинное обучение / С. Рашка, В. Мирджали; пер. с англ. — 3-е изд. — Москва: ДМК Пресс, 2020. — 418 с.
13. Технологии командной разработки приложений: методические указания к выполнению курсовой работы / Белорусско-Российский ун-т, каф. программного обеспечения информационных технологий. — Могилёв: БРУ, 2022. — 20 с.

## Приложение 1 – Код программных модулей